

基于交会定位与滚动覆盖调度的无线电干扰源自动清除模型

摘要

无线电干扰源的快速定位与清除是频谱管理的重要任务。本文针对测向误差、源数未知及接收范围受限下的定位区域计算、二次测点选择和机器狗多源搜索清除问题，综合建立测向楔形交会、多指标选点及认证搜索—保守定位—滚动清除模型，并基于半平面求交、旋转卡壳、两级搜索与启发式路径优化方法求解。

针对问题一中示向度存在 $\pm 1^\circ$ 误差条件下的干扰源定位问题，本文将单次测向表示为闭楔形并转化为线性半平面约束，建立多楔形交会定位模型。通过半平面交构造凸定位区域，采用旋转卡壳算法求取区域直径及最远点对，并检验同直径圆的覆盖性。100 组随机算例均成功定位；等边反例 $D \approx 20\text{ m}$ ，最小覆盖半径为 $11.547\text{ m} > D/2$ ，故同直径圆不能覆盖该区域。

针对问题二，考虑第二检测点选择中定位精度、接收可靠性与移动代价的冲突，构建物理可行域，建立兼顾几何质量、保证接收比例、移动时间和定位误差代理的四指标选点模型，采用全局粗搜—局部加密和蒙特卡洛权重校准求解。在 40 个合成回放场景中，二测接受率为 100%，平均实际 MEC 为 42.62 m，二测移动与检测平均耗时 109 s，整体优于对照策略，并以 $\text{MEC} \leq 20\text{ m}$ 作为后续清除判据。

针对问题三，面向 10—16 个数量未知的全向干扰源，建立覆盖—定位协同滚动调度模型。利用原点与 1000 m 七点环保障全域发现，以 $\pm 1^\circ$ 方位带构造保守可行域，结合四指标选点、共观测和逐动作重规划完成清除。离线 3000 局全清除，平均耗时 293.5 s/源；三次正式测试分别清除 11、13、14 个，后两局加权平均耗时 272.112 s/源；第一局因日志写入异常中止，但前 11 个源清除记录完整，可用于反应平均清除时间。

针对问题四，主要解决全向与定向干扰源并存时的可靠发现与清除问题。通过分析方向约束与无信号多义性，建立认证覆盖、路线门控、保守定位和安全剪枝相结合的 D 模型。题设内 6150 次鲁棒性运行全部清除，三次正式测试分别实现 12/12、16/16 和 13/13 全清除。三次正式测试以各次清除的干扰源数量为权重，加权平均清除时间为 441.018 s/源。

最后，本文对模型进行鲁棒性检验。在多随机种子、多类压力情景及不同干扰源数量、分布位置、发射方向和测向误差条件下开展的大规模仿真实验中，模型的干扰源清除率均达到 100%，表明其在复杂不确定环境下具有良好的鲁棒性与可靠性。

关键词：半平面交会定位 多指标选点模型 滚动搜索清除

定向覆盖认证模型

1 问题重述

1.1 问题背景

无线电频谱是通信、导航和应急保障系统正常运行的重要资源。异常发射设备形成的无线电干扰可能降低通信质量，甚至造成业务中断，因此，快速定位并清除干扰源是频谱管理的重要任务。传统精准排查主要依靠技术人员携带测向设备进入目标区域，根据不同位置的信号场强和方位信息逐步逼近目标；面对环境遮挡、低功率信号和危险作业条件时，该方式耗时较长且存在安全风险。为提高处置效率，本文研究搭载测向机、光学探测仪和清除装置的机器狗系统。研究对象为半径 1800 米圆形区域内频道各异、数量未知的全向与定向干扰源，重点讨论测向误差下的定位区域计算、检测点选择，以及多干扰源的自主搜索、精准定位和清除。

1.2 问题提出

问题一： 根据若干检测点的坐标及其对应示向度，建立多边形定位区域直径的计算方法，并判断以该直径为直径的圆能否完全覆盖定位区域。

问题二： 根据某全向干扰源在首个检测点处的示向度，制定第二个检测点的选择策略，并得到定位效果较好的候选区域。

问题三： 针对目标区域内数量、频道及位置未知的全向干扰源，设计机器狗自动搜索、定位和清除策略，在确保全部干扰源被清除的条件下尽可能缩短任务时间，并通过演练及三次正式测试检验算法有效性。

问题四： 针对目标区域内全向与定向干扰源混合分布，且干扰源类型、数量、位置及定向方向均未知的情形，改进机器狗的检测与清除策略，并通过模拟测试评价算法的有效性。

2 问题分析

2.1 子问题分析

2.1.1 问题一分析

问题一要求基于多个检测点的示向度信息，在每次测向误差不超过 1° 的条件下确定干扰源的可能位置范围，并分析该区域的最大尺度及其圆覆盖性质。由于示向度存在误差，单次观测不能表示为一条确定射线，而应表示为以检测点为顶点、张角为 2° 的可行楔形区域。因此，本文采用计算几何中的半平面交方法：将每条楔形的两条边界分别转化为线性半平面约束，取全部约束的交集作为干扰源的保守定位区域。

进一步地，该交集为凸集；当其非空且有界时可表示为凸多边形。利用凸多边形直径必由一对顶点取得的性质，采用旋转卡壳算法计算区域内最远点对及区域直径，并以全顶点对枚举进行校验。最后，以最远点对中点构造半径为 $D/2$ 的同直径圆，检查其是否包含全部顶点，从而判断其能否覆盖整个定位区域。

2.1.2 问题二分析

问题二要求在首次测向结果已知的基础上选择第二检测点，使定位区域尽可能小，同时兼顾接收可靠性与移动时间。首先，将首次示向楔形、干扰源活动范围和有效接收距离相交，得到初始可行域；随后在域内采样可能源位置，并通过“全局粗搜索—局部加密”生成候选检测点。对各候选点计算交会几何质量、保证接收比例、移动时间和预测定位半径，归一化后建立加权评价模型。权重由蒙特卡洛模拟和网格搜索确定，再将得分接近最优值的候选点组成较优区域，选择其中移动时间最短者执行检测。

2.1.3 问题三分析

问题三要求在干扰源数量、频道和位置均未知时完成全域搜索、精确定位与可靠清除，并尽量缩短任务时间。本文建立多频道状态驱动的协同搜索模型：利用示向误差带、接收距离和半平面交动态更新各频道的保守可行域，以最小覆盖圆建立可靠清除判据；综合几何互补性、接收可靠性、预计完成时间和预测定位半径选择观测点；联合优化覆盖环点数与半径，平衡全域发现能力、扫描次数和移动距离。在多源调度中加入预算内共观测、顺路清除和开放路径优化，并在每次测量或清除后更新状态、滚动规划。采用瓶颈驱动迭代求解，通过配对试验筛选机制，并结合覆盖证明、消融试验和多种子仿真验证模型。

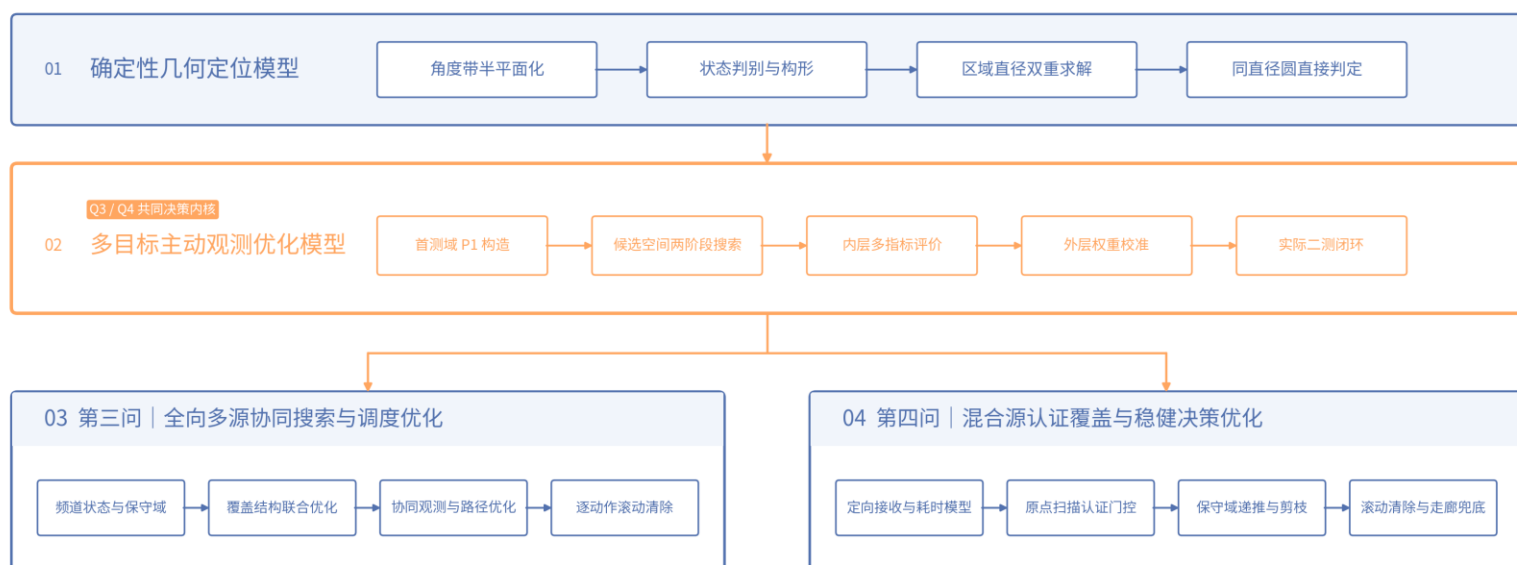
2.1.4 问题四分析

问题四在第三问框架中进一步考虑定向干扰源的发射方向约束。由于未接收信号可能由距离过远或机器狗处于发射背面造成，不能据此直接排除邻近区域。为此，建立包含接收距离、发射半平面和示向误差的定向传播模型，并设计适用于边界困难场景的稳健路线 B 和常规场景的高效路线 C，根据原点扫描结果选择路线。对已发现频道，将作业圆域、示向角带和最大接收圆求交形成保守可行域，再以候选测站到可行域的最小距离筛除必然无效的共观测点。最后采用多方位定位和滚动调度，并设置示向走廊点阵作为困难情形下的兜底方案，在保证全部清除的基础上优化平均耗时与尾部耗时。

2.2 总体思路

为更加直观、清晰地展示本文所构建的模型及其整体框架，绘制模型框架图如下：

图 1-1 整体建模流程图



3 模型假设

1. 观测期间各辐射源的位置保持不变，测量站与辐射源均可视为平面内的点，且测量站坐标准确。
2. 每个辐射源占用唯一且固定的工作信道，不同辐射源之间相互独立，不考虑同频干扰。
3. 测向误差限定在 $\pm 1^\circ$ 内；测量站位置不变时，对同一辐射源重复测向不改变既有误差。
4. 任务期间接收半径固定为 1000~1500 m，忽略遮挡、多径等附加影响。
5. 机器人以 5 m/s 匀速直线运动，移动时不探测；各项耗时按题设，计算与通信时间不计。

4 符号说明

表 4-1 主要符号说明

符号	说明	单位
$S/X = (x, y)$	信号源位置坐标（问题一为 S，其余为 X）	m
$M_i = (x_i, y_i)$	第 i 个检测点的位置坐标	m
θ_i	第 i 次有效检测得到的示向度	°
ε	示向度误差半角（本文取 1°）	°
A_R	半径为 R 的圆形作业区域	—
$P_h^{(k)}$	频道 h 经 k 次有效示向更新后的保守定位可行域	—
v_j	凸定位多边形的第 j 个顶点坐标	m
D	定位可行域的直径（区域内最大点间距离）	m
r_h	频道 h 对应信号源的实际接收半径	m
$c_{MEC,h}^{(k)}$	可行域最小覆盖圆的圆心	m
$r_{MEC,h}^{(k)}$	可行域最小覆盖圆的半径	m
Q_{geo}	候选检测点的交会几何质量指标	—
P_{rec}	候选检测点的信号接收比例（或接收概率）	%
R_{pred}	新增观测后的预测定位半径	m
$J(q)$	候选检测点 q 的四指标综合评分	—
z_h	频道 h 的在线任务状态向量	—
R_{cov}	全域搜索中等角外环的覆盖半径	m
β_h	频道 h 对应定向源的发射中轴方向	°
L	机器狗执行任务的总移动距离	m
N_{switch}	频道切换的次数	次
$N_{measure}$	频道测向的次数	次
N_{clear}	频道清楚的次数	次
T	任务总虚拟时间	s

5 模型的建立与求解

5.1 问题一的模型

5.1.1 模型建立

(1) 符号约定

记干扰源位置为 $S = (x_S, y_S)$ ，第 i 个检测点为 $M_i = (x_i, y_i)$ ，其有效示向度为 $\hat{\theta}_i$ ，角度从正东方向开始逆时针计量。记误差半角为 $\varepsilon = 1^\circ$ ，第 i 条观测对应的闭楔形为 W_i ，全部楔形的交集为 P 。当 P 为有界凸多边形时，其逆时针顶点依次记为 $v_0, v_1, \dots, v_{m-1}$ ，区域直径记为 D 。本问仅利用角度信息构造保守定位区域，不将半径为 1800 m 的目标圆域加入主模型，以避免圆域裁剪产生圆弧边界。

(2) 第一步：将角度误差带转化为半平面

第 i 条示向度误差带的下、上边界方向向量分别为

$$\mathbf{u}_i^- = (\cos(\hat{\theta}_i - \varepsilon), \sin(\hat{\theta}_i - \varepsilon)) \quad (1-1)$$

$$\mathbf{u}_i^+ = (\cos(\hat{\theta}_i + \varepsilon), \sin(\hat{\theta}_i + \varepsilon)) \quad (1-2)$$

定义二维叉积 $\text{cross}(\mathbf{a}, \mathbf{b}) = a_x b_y - a_y b_x$ 。对任意候选点 $X = (x, y)$ ，其位于第 i 个测向楔形内的充要条件为

$$W_i = \{X \mid \text{cross}(\mathbf{u}_i^-, X - M_i) \geq 0, \text{cross}(\mathbf{u}_i^+, X - M_i) \leq 0\} \quad (1-3)$$

式 (1-3) 中的第一个不等式要求候选点位于下边界的左侧，第二个不等式要求其位于上边界的右侧，二者共同选出朝向干扰源的闭楔形。向量表达不需要直接比较角度大小，因而能够自然处理示向度跨越 $0^\circ/360^\circ$ 的情况。

将式 (1-3) 展开，可将两侧约束统一写成 $a_j x + b_j y \leq c_j$ 的线性半平面形式：

$$u_{iy}^- x - u_{ix}^- y \leq u_{iy}^- x_i - u_{ix}^- y_i \quad (1-4)$$

$$-u_{iy}^+ x + u_{ix}^+ y \leq -u_{iy}^+ x_i + u_{ix}^+ y_i \quad (1-5)$$

因此， n 条有效示向度共形成 $2n$ 个闭半平面约束。

(3) 第二步：定义定位区域并证明其凸性

全部观测的共同可行域定义为

$$P = \bigcap_{i=1}^n W_i = \{X \mid AX \leq b\} \quad (1-6)$$

逐次加入观测时有 $P_k = P_{k-1} \cap W_k$ ，故新增有效观测只会使定位区域缩小或保持不变。每个线性半平面均为凸集，每个楔形 W_i 是两个半平面的交，有限个凸集的交仍为凸集，因此定位区域 P 必为凸集。当 P 非空、有界且具有正面积时， P 为凸多边形；特殊情况下也可能为空集、无界区域、线段或单点。

(4) 第三步：构造定位多边形

首先检查半平面组的可行性。程序依次考察原点、原点到各边界直线的垂足以及任意两条非平行边界直线的交点，并将候选点代入全部约束。若不存在满足所有约束

的点，则判定 $P = \emptyset$ ，返回空集状态。此时应检查观测归属、角度单位、检测点坐标和误差界。

若区域非空，则进一步检查有界性。设第 j 个半平面的法向量为 $\mathbf{n}_j$ ，区域无界当且仅当存在非零方向 $\mathbf{d}$ ，使

$$\mathbf{n}_j \cdot \mathbf{d} \leq 0, \quad \forall j \quad (1-7)$$

式 (1-7) 表明，从任一可行点沿 $\mathbf{d}$ 方向可以无限延伸而不违反约束。二维非平凡衰退锥必包含某条约束边界的切向方向，因此程序枚举各边界的两个切向方向进行检查。若找到满足式 (1-7) 的方向，则返回无界状态、一个可行点和一个非零延伸方向，区域直径记为正无穷。此时不能用人为设置的大矩形截断区域，否则所得直径依赖人为边界，并非原定位区域的真实直径。

对于非空且有界的区域，程序枚举任意两条非平行边界直线的交点，仅保留满足全部半平面约束的交点，再进行精确去重并采用单调链算法求凸包，删除共线边上的冗余中间点，最终获得逆时针排列的顶点序列。

上述定位区域的几何构造过程如图 5-1 所示。图 5-1(a)中，虚线表示各检测点的测量示向线，其两侧实线为示向度误差界对应的楔形边界；不同测向楔形相交后，在真实源点 S 附近形成有限可行域。图 5-1(b)进一步展示了半平面边界交点构成的凸定位多边形，其中连接最远顶点对的线段长度即为区域直径 D 。

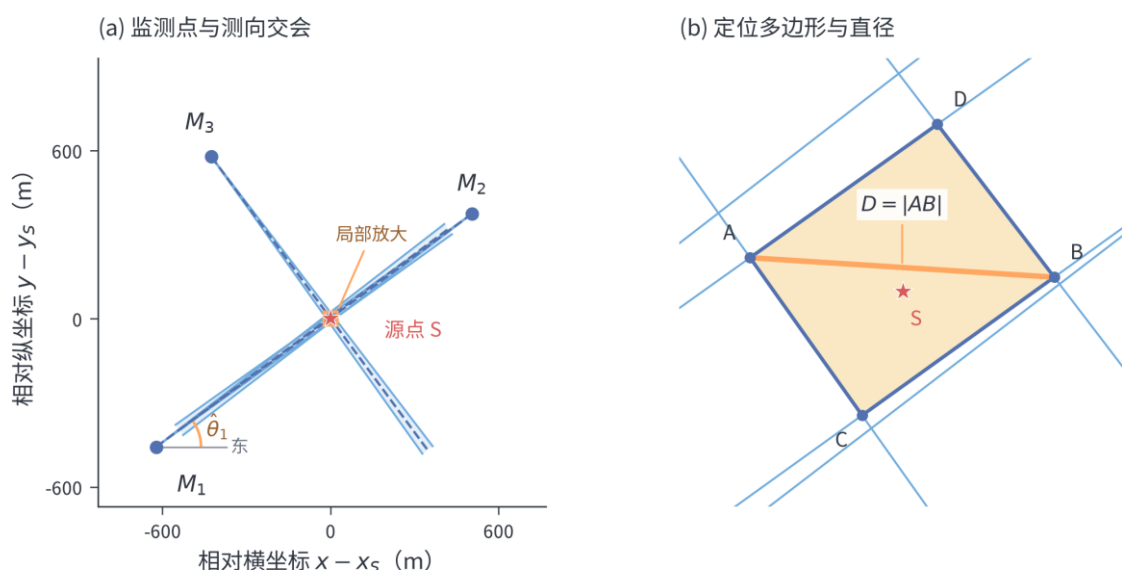


图 5-1 示向度交会定位区域及区域直径示意图

(5) 第四步：计算区域直径

如图 5-1(b)所示，半平面交的可行边界形成凸定位多边形，区域内距离最远的两个顶点构成最远点对。为定量描述定位区域的最大空间跨度，将区域直径定义为区域内任意两点之间的最大距离：

$$D = \max_{p, q \in P} \|p - q\|_2 \quad (1-8)$$

由于 P 为凸多边形，任意 $p, q \in P$ 均可写成顶点的凸组合 $p = \sum_i \alpha_i v_i$ 、 $q = \sum_j \beta_j v_j$ ，其中 $\alpha_i, \beta_j \geq 0$ 且各自之和为 1。由三角不等式可得

$$\|p - q\|_2 = \left\| \sum_{i,j} \alpha_i \beta_j (v_i - v_j) \right\|_2 \leq \sum_{i,j} \alpha_i \beta_j \|v_i - v_j\|_2 \leq \max_{i,j} \|v_i - v_j\|_2 \quad (1-9)$$

顶点本身属于 P ，因此区域直径必由某对顶点取得：

$$D = \max_{0 \leq i < j \leq m-1} \|v_i - v_j\|_2 \quad (1-10)$$

程序默认采用旋转卡壳算法求凸多边形的对踵点。程序另用复杂度为 $O(m^2)$ 的全顶点对枚举进行交叉验证，要求两种算法所得最大距离平方精确一致。空集没有可用直径，无界区域直径为正无穷；单点与线段是合法退化结果，其直径分别为 0 和两端点距离。

(6) 第五步：判断同直径圆能否覆盖定位区域

本问所称的圆是直径恰等于区域直径 D 的同直径圆。设 a, b 为一对最远点，满足 $\|a - b\|_2 = D$ 。若半径为 $D/2$ 的圆能够同时包含 a, b ，设圆心为 c ，则

$$D = \|a - b\|_2 \leq \|a - c\|_2 + \|c - b\|_2 \leq \frac{D}{2} + \frac{D}{2} = D \quad (1-11)$$

式 (1-11) 中的两个不等式必须同时取等号，故 a, b, c 共线，且圆心只能为最远点对的中点：

$$c = \frac{a + b}{2} \quad (1-12)$$

因此不必搜索其他圆心，只需检验全部顶点是否满足

$$\max_{0 \leq i \leq m-1} \|v_i - c\|_2 \leq \frac{D}{2} \quad (1-13)$$

圆盘是凸集，因此包含全部顶点就等价于包含整个凸多边形。若存在顶点不满足式 (1-13)，则不存在任何直径为 D 的圆可以覆盖该定位区域。

为判断上述覆盖关系是否对所有定位区域都成立，考虑边长为 D 的等边三角形。其区域直径为 D ，而最小外接圆半径为

$$R_{\min} = \frac{D}{\sqrt{3}} > \frac{D}{2} \quad (1-14)$$

因此，任何直径为 D 的圆都无法覆盖该等边三角形。该三角形可以由三组 $\pm 1^\circ$ 测向楔形的交构造出来，属于本模型允许的定位区域；另一方面，矩形可以被以其对角线为直径的圆覆盖。由此可得，同直径圆不一定能够覆盖定位区域。式 (1-14) 中的最小外接圆只用于解析验证该反例，给定一般定位区域时仍采用式 (1-12) 和式 (1-13) 直接判定，不需要计算通用最小外接圆。

综上，问题一的建模过程依次完成了示向度半平面化、凸定位区域构造、区域直径计算和同直径圆覆盖判定，从而将带角度误差的交会定位问题转化为可直接求解的凸计算几何问题。针对问题一整体建模流程如下图：



图 5-2 问题一建模流程图

5.1.2 求解与结果分析

根据前述模型，求解程序依次完成半平面生成、区域状态判别、可行顶点筛选、凸包构造、区域直径计算和同直径圆覆盖判定。

(1) 定位区域求解

输入检测点坐标与对应示向度后，每条观测产生两个闭半平面。程序首先检查约束组的可行性和有界性，再枚举边界交点并筛除不满足全部约束的点，最后对可行交点求凸包。若区域状态为 有界凸多边形，即可获得干扰源的有限定位区域；若状态为空集，说明观测之间存在矛盾；若状态为 无界区域，说明现有检测方向不足，需要补充不同方位的有效观测。

半平面交采用边界交点枚举，复杂度为 $O(k^3)$ ，其中 $k = 2n$ 为半平面数量；凸包预处理复杂度为 $O(m \log m)$ ，旋转卡壳求直径的复杂度为 $O(m)$ 。由于本问的有效观测数量较少，该算法能够在保持几何含义清晰的同时完成稳定求解。

(2) 区域直径求解与定位精度分析

对所得凸定位区域，旋转卡壳算法确定一对最远顶点 a, b ，并计算区域直径 $D = \|a - b\|_2$ 。程序同时采用全顶点对枚举进行复核，两种方法所得最大距离平方完全一致。区域直径给出了两个可行位置之间的最大距离，可作为定位不确定性的尺度： D

越小，说明各测向楔形的交会约束越强； D 较大通常意味着检测点方位分布较差或测向边界的交会角较小。

因此，在实际检测中，仅增加与已有测向方向近似平行的观测，可能难以显著缩小定位区域；优先选择侧向位置补充测向，使新增楔形从不同方向切割原区域，通常更有利于减小区域直径。

(3) 同直径圆覆盖结果分析

由最远点对可知，若存在直径为 D 的覆盖圆，其圆心只能取 $c = (a + b)/2$ ，半径为 $D/2$ 。将全部区域顶点代入覆盖判据即可得到确定结论：若所有顶点均位于圆内，则该圆覆盖整个凸区域；若任一顶点位于圆外，则不存在其他同直径圆能够覆盖该区域。

为判断同直径圆是否必然覆盖定位区域，程序构造了一个可由三组 $\pm 1^\circ$ 测向楔形交会形成的等边三角形。该算例的关键结果如表 5-1 所示。

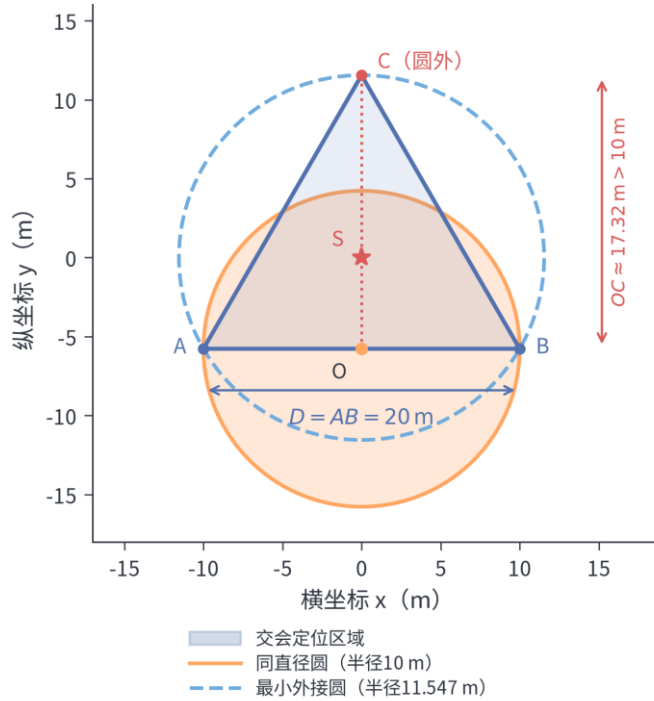
表 5-1 等边三角形反例的关键计算结果

指标	理论值	计算值	判定
三角形边长	20 m	—	—
区域直径 D	20 m	19.99999999999977 m	一致
同直径圆半径 $D/2$	10 m	9.99999999999989 m	一致
最小外接圆半径	$20/\sqrt{3} \approx 11.547005$ m	—	$> D/2$
同直径圆覆盖性	不能覆盖	不能覆盖	一致

由表 5-1 可知，等边三角形的区域直径约为 20 m，相应同直径圆半径约为 10 m；但该三角形的最小外接圆半径为 $20/\sqrt{3} \approx 11.547005$ m，大于 $D/2$ ，故同直径圆无法覆盖该区域。另一方面，矩形可以被以其对角线为直径的圆覆盖。因此，本问的最终结论是：以定位区域直径为直径的圆不一定能够覆盖定位区域。

图 5-3 等边三角形定位区域与同直径圆覆盖反例图

等边三角形反例：同直径圆不能覆盖定位区域



$$R_{\min} = 20/\sqrt{3} \approx 11.547\text{ m} > D/2 = 10\text{ m}$$

算例 EQ_LAST；S 为真实源点及本例最小外接圆圆心，O 为 AB 中点。

综上，所建模型不仅能够给出干扰源的可行定位区域及其最大空间跨度，还能对给定区域的同直径圆覆盖性作出直接判定。该结论说明，区域直径只能描述最大点间距离，不能单独保证存在半径为 $D/2$ 的统一覆盖位置。

5.1.3 模型检验

（1）定位正确性与直径算法交叉验证

为检验模型能否在误差界内保留真实干扰源，采用固定随机种子 20260911 生成 100 组蒙特卡洛算例。每组在半径 1800 m 的圆盘内随机生成真实源点，并在距源点 $50\sim 900\text{ m}$ 的环域内随机生成 $2\sim 6$ 个检测点；随后在真实方位上叠加不超过 $\pm 1^\circ$ 的模拟误差。对每组算例，分别检查输入误差是否满足上界、真实源点是否满足全部半平面约束并位于输出定位区域内、输出顶点是否满足全部约束。结果表明，100 组算例均通过上述检查，且均得到有界凸多边形。

同时，程序以全顶点对枚举结果作为旋转卡壳算法的独立参照。100 组算例中，两种方法得到的最大距离平方均完全一致，说明区域直径及最远点对的计算结果可靠。需要指出的是，随机实验仅用于发现实现错误，不能替代定位区域凸性、顶点取径和圆覆盖判定的理论证明。

（2）边界情形与敏感性检验

为避免连续随机抽样难以覆盖退化构型，进一步设置了空集、无界、单点、线段、近乎平行边界、角度跨 0° 、小交会角、相背观测和单次测向无界共 9 类确定性测试。结果均返回预期状态：矛盾约束被识别为空集，方向信息不足时返回无界状态，单点和线段被保留为合法退化结果，近乎平行边界也未被错误删除。9 项测试全部通过。

此外，固定同一组检测点和示向度，将误差半角依次设为 0.9° 、 1.0° 和 1.1° ，检验误差界放宽后定位区域的包含关系及有限区域直径的单调性。共完成 300 项检查，全部通过；其中 299 项得到凸多边形，1 项在 0.9° 误差界下为空集。该空集是因为模拟误差允许落在 0.9° 以上，属于预期现象，而非模型失效。

(3) 反例检验与模型适用性

针对同直径圆覆盖命题，构造边长约 20 m 的等边三角形定位区域。计算得到的区域直径与理论边长之差小于 10^{-8} m，同时确认半径为直径一半的圆无法覆盖该区域；该结果与等边三角形最小外接圆半径大于半径 $D/2$ 的理论结论一致。由此验证了“不一定能够覆盖”的反例构造及判定程序。

综上，模型在常规随机算例、边界退化构型和误差界扰动下均表现稳定。其适用前提是检测点坐标准确、观测对应同一固定干扰源，且有效示向度误差满足给定上界。由于普通角度方向仍由三角函数计算，极端临界构型可能受到浮点舍入影响；当输出为空集或无界区域时，应结合观测质量和检测点方位重新布设或补充观测，而不应通过人为截断或放宽误差界得到表面上的有限结果。

5.2 问题二的模型

问题二要求依据有限角度与距离信息确定下一测量点，以较少飞行与测量代价缩小目标可能区域。本文将该任务表述为带物理约束的序贯决策问题：先构造目标可行域，再以几何收缩、接收概率、运动代价和方向互补性评价候选点，最后通过最小覆盖圆判定是否满足搜寻精度。

5.2.1 模型建立

(1) 目标物理可行域

设第 k 次测量点为 S_k ，测得方位角为 θ_k ，角度误差上界为 ε ，目标与测量点距离满足 $d_{\min} \leq d \leq d_{\max}$ ，目标同时位于半径 R 的作业圆内。由角度带、距离环带和作业圆的交集得到第 k 次测量后的可行域：

$$P_1 = \{S \mid |\text{wrap}(\theta(S, M_1) - \hat{\theta}_1)| \leq \varepsilon, \|S\|_2 \leq 1800, 5 < \|S - M_1\|_2 \leq 1500\}. \quad (2-1)$$

首次测量形成的扇环形区域如图 5-4 所示。该区域给出了目标的全部物理解，后续搜索只需在此区域内进行，从而避免在整个作业圆中盲目采样。

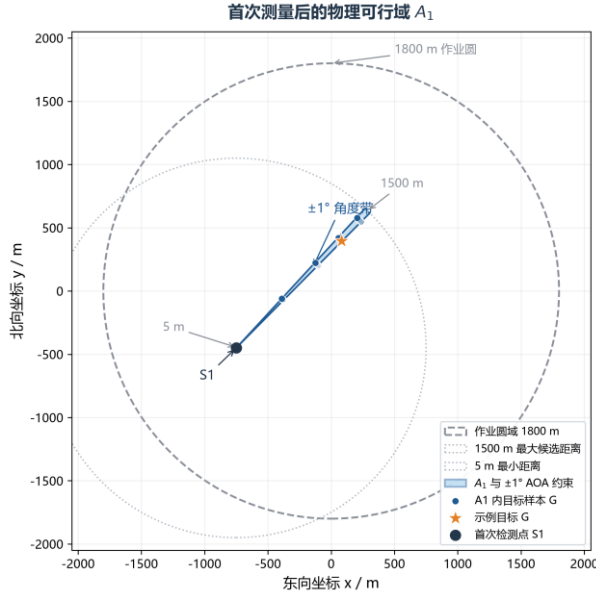


图 5-4 首次测量后的物理可行域

(2) 候选点生成与四指标评价

单移动平台可以根据已有方位观测主动选择下一观测位置，从而逐步降低目标位置的不确定性[1]。据此，本文以当前测量点为极点，在允许航程内采用“全局粗搜索—局部加密”的离散主动选点方法：首先在半径和方位方向均匀生成候选点，保留作业圆内且满足最小位移约束的点；随后围绕粗搜索所得高分点缩小搜索步长，得到最终候选集 C_k 。对任一候选点 S ，计算以下四项评价指标。

$$Q_{\text{geo}}(M_2) = \frac{1}{N} \sum_{j=1}^N \sin^2 \alpha_j \cdot P_{\text{rec}}(M_2) = \frac{1}{N} \sum_{j=1}^N \mathbb{I} \left(\|S^{(j)} - M_2\|_2 \leq 1000 \right), \quad (2-2)$$

其中， Q_{geo} 以两观测方向夹角的平方正弦刻画几何互补性， P_{rec} 统计候选点处样本落入有效接收半径的比例。运动代价采用归一化飞行时间，预测定位半径 R_{pred} 则综合有效样本的前向误差：

Bearing-only 定位精度与观测点的几何构型密切相关；当两条观测方向接近正交时，方位误差引起的定位误差放大效应通常较小[2]。AOA 交叉定位的 GDOP 分析同样表明，测向站位分布及交会角是影响定位精度的重要因素[4]。因此，本文以两观测方向夹角的平方正弦定义几何质量指标 Q_{geo} ，用于刻画候选点的几何互补性。该指标仅用于候选点评分，实际定位区域仍根据题设给定的 $\pm 1^\circ$ 角度带计算。与此同时， P_{rec} 表示候选点处样本落入有效接收半径的比例，运动代价采用归一化飞行时间，预测定位半径 R_{pred} 则综合有效样本的前向误差：

$$T_{\text{move}}(M_2) = \frac{\|M_2 - M_1\|_2}{5} \cdot R_{\text{pred}}(M_2) = \frac{\sum_{j=1}^N \mathbb{I}_j R_{\min}(P_2^{(j), \text{pred}})}{\sum_{j=1}^N \mathbb{I}_j}, \quad \mathbb{I}_j \quad (2-3)$$

$$= \mathbb{I}(\|S^{(j)} - M_2\|_2 \leq 1000).$$

在存在测角误差和接收范围约束的情况下，若仅追求最大的交会角，可能造成移动距离过大或第二次检测无法接收到有效信号。谨慎贪心主动定位方法强调同时权衡定位不确定性的下降程度与移动代价[3]。据此，本文将几何质量、保守接收比例、移动时间和预测定位半径分别归一化后构造综合评分函数。综合评分越大，说明候选点在几何互补性、通信可靠性、飞行代价和预测定位误差之间的综合表现越优：

$$J_0(M_2) = w_1 \tilde{Q}_{\text{geo}} + w_2 \tilde{P}_{\text{rec}} - w_3 \tilde{T}_{\text{move}} - w_4 \tilde{R}_{\text{pred}}, \quad w_i \geq 0, \quad \sum_{i=1}^4 w_i = 1. \quad (2-4)$$

为降低离散峰值对网格扰动的敏感性，先取接近最大评分的候选点组成高分区域 R_{good} ，再从中选择运动代价最小的点作为下一执行点：

$$\mathcal{R}_{\text{good}} = \{M_2 \in \mathcal{C}_{\text{ref}} \mid J(M_2) \geq (1 - \eta)J^*\}. M_2^* = \underset{M_2 \in \mathcal{R}_{\text{good}}}{\operatorname{argmin}} T_{\text{move}}(M_2), \quad (2-5)$$

图 5-5 展示了评分峰值、高分候选区域与最终执行点的位置关系。二阶段选择保留了高评分候选的定位收益，同时减少了不必要的飞行。

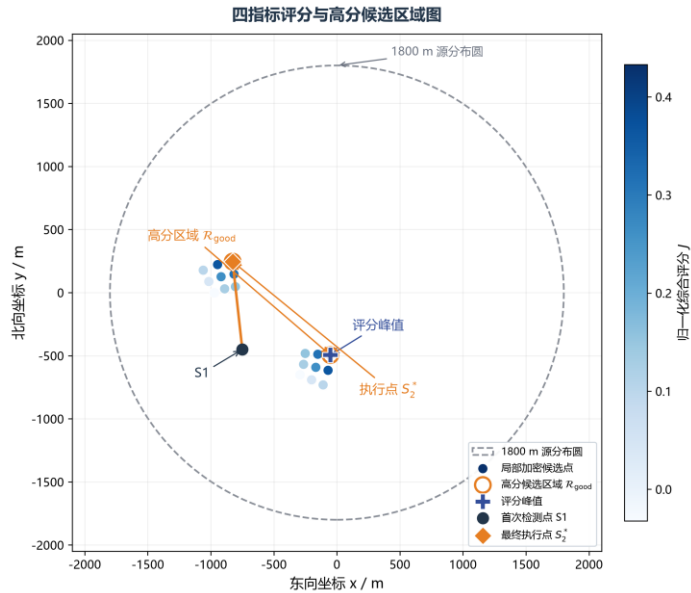


图 5-5 四指标评分与高分候选区域

(3) 状态更新与停止准则

无人机到达 S_{k+1}^* 后，根据是否接收到信号更新可行域：有信号时叠加新的角度带与距离约束，无信号时剔除接收半径内区域。以更新后可行域的最小覆盖圆半径 r_{MEC} 作为定位误差上界；当 $r_{\text{MEC}} \leq 20 \text{ m}$ 时停止搜索，否则进入下一轮。

$$P_1 \leftarrow P_1 \cap \{S \mid \|S - M_2^*\|_2 > 1000\}, \quad (2-6)$$

5.2.2 求解与结果分析

(1) 权重标定与单场景求解

在训练场景上对权重进行网格搜索，以接收失败、覆盖圆半径和飞行时间的加权损失为目标。代表性最优权重为 $w^*=(0,0.25,0.50,0.25)$ ，对应训练集接收率 100%，最小覆盖圆半径均值 46.04 m、中位数 37.23 m、90% 分位数 83.18 m，20 m 内成功率为 25%，平均总耗时为 91 s。结果说明在当前候选生成方式下，运动代价与方向互补性比单步几何收缩更能稳定改善总体效率。

(2) 策略对比

为检验联合策略的有效性，将其与仅几何、最近点和随机策略在同一测试集上比较，结果见表 5-2。联合策略实现 100% 接收率，并取得最小的覆盖圆半径和最高的 20 m 内成功率；最近点策略虽然耗时最短，但观测方向高度相关，定位误差最大；随机策略性能波动明显。

表 5-2 不同选点策略的测试结果

策略	接收率	MEC 均值/m	MEC 中位数/m	$\leq 20 \text{ m}$ 成功率	平均总耗时/s
四指标联合	100.0%	42.62	38.95	35.0%	109
仅几何	0.0%	250.00	250.00	0.0%	361
最近点	100.0%	437.66	484.26	0.0%	15
随机	67.5%	187.04	198.71	2.5%	181

综合来看，联合策略并非只追求单次几何收益，而是以适度飞行换取稳定接收和方向互补，因此在定位精度与任务耗时之间取得了更合理的平衡。

5.2.3 模型检验

(1) 边界与接收逻辑检验

选取近似平行观测、精度阈值附近、明显满足阈值和明显不满足阈值四类边界场景，并对接收概率的内区、过渡区和外区进行核验。表 5-3 表明，最小覆盖圆判据在 20 m 阈值附近能够正确切换，接收逻辑也随距离单调变化。

表 5-3 边界与接收逻辑检验结果

检验类型	条件	结果	判定
定位边界	近似平行观测	$r_{\text{MEC}}=700.27 \text{ m}$	继续
定位边界	阈值附近	$r_{\text{MEC}}=21.83 \text{ m}$	继续
定位边界	明显满足	$r_{\text{MEC}}=13.97 \text{ m}$	停止
定位边界	明显不满足	$r_{\text{MEC}}=112.48 \text{ m}$	继续
接收逻辑	距离 900 m	接收率 100.0%	正确
接收逻辑	距离 1200 m	接收率 60.45%	正确
接收逻辑	距离 1600 m	接收率 0.0%	正确

(2) 权重敏感性分析

分别将各权重在基准值附近扰动 $\pm 10\%$ ，并重新归一化。除 w_2 下调 10% 时接收率由 100% 降为 0%、20 m 内成功率下降 5 个百分点外，其余扰动下主要结果保持不变； w_2 下调时 MEC 均值反而下降 0.92 m，说明较小的几何改善不足以补偿通信失效。基准权重 $w_1=0$ ，因此其相对扰动不改变策略。由此可见，模型对 w_3 、 w_4 的小幅变化较稳健，而接收概率权重 w_2 存在明确下限，应优先保证通信可达性。

综上，边界判据、接收逻辑与敏感性结果相互一致，模型能够稳定执行“生成候选—综合评分—状态更新—精度判停”的闭环流程。

5.3 问题三模型

5.3.1 模型建立

(1) 任务状态与保守定位模型

设机器狗总移动距离为 L ，频道切换、测向、失败清除和成功清除次数分别为 N_{switch} 、 N_{measure} 、 $N_{\text{clear,fail}}$ 和 $N_{\text{clear,success}}$ 。按照题目规定的动作耗时，总虚拟时间为

$$T = \frac{L}{5} + N_{\text{switch}} + 5N_{\text{measure}} + 3N_{\text{clear,fail}} + 5N_{\text{clear,success}}. \quad (3-1)$$

模型首先保证所有信号源均被可靠清除，再依次减小平均每源总耗时与 P95 每源总耗时。对频道 $h \in 1, \dots, 20$ ，维护在线状态 $\mathbf{z}_h = (\sigma_h, \mathcal{O}_h, \mathcal{N}_h, P_h, c_{\text{MEC},h}, r_{\text{MEC},h})$ 。其中， σ_h 为未知、已发现、已清除或已排除状态， $\mathcal{O}_h$ 和 $\mathcal{N}_h$ 分别为有效示向观测与无信号检测点集合， $c_{\text{MEC},h}$ 、 $r_{\text{MEC},h}$ 为可行域 P_h 的最小覆盖圆圆心和半径。只有清除操作返回成功时才将频道标记为已清除。

设频道 h 在检测点 M_i 得到示向度 θ_i ，误差上界为 $\varepsilon = 1^\circ$ 。沿用问题一的半平面交方法，以 B_i^{angle} 表示示向误差带，以 B_i^{range} 表示 1500 m 最大接收距离约束，记作业圆域为 $\mathcal{A}_R = \{X: \|X\|_2 \leq R\}$ ，则经过 k 次有效观测后的保守可行域为

$$P_h^{(k)} = \mathcal{A}_R \cap \bigcap_{i=1}^k (B_i^{\text{angle}} \cap B_i^{\text{range}}). \quad (3-2)$$

无信号响应会形成半径 1000 m 的非凸排除孔。实现中仅删除排除圆内的信念样本，清除判定仍依据 $P_h^{(k)}$ 的保守外包络。考虑数值误差并预留 $\delta = 0.25 \text{ m}$ ，最小覆盖圆及可靠清除条件为

$$\begin{aligned} \text{MEC}(P_h^{(k)}) &= (c_{\text{MEC},h}^{(k)}, r_{\text{MEC},h}^{(k)}), \\ r_{\text{MEC},h}^{(k)} &\leq 20 - \delta = 19.75 \text{ m}. \end{aligned} \quad (3-3)$$

满足式 (3-3) 时，机器狗前往 $c_{\text{MEC},h}^{(k)}$ 清除；否则继续补充观测。该判据以可行域整体而非单个估计点为依据，可以避免误清除。

(2) 多源在线测量决策模型

对尚不能清除的频道，模型沿用问题二的“全局粗搜索—局部加密—高分区域择近”框架：在距离 200、600、1000、1400、1800 m 与 60° 方位步长上粗搜，再围绕前两名候选点按距离 $\pm 100 \text{ m}$ 、方位 $\pm 10^\circ$ 加密。相关干扰源定位研究表明，观测点与目标之间的几何构型以及观测点的顺序配置会共同影响定位精度和路径执行效率[6]。因此，本文在问题二选点模型的基础上，仍以几何互补性 Q_{geo} 、接收可靠性 P_{rec} 和预测定位半径 R_{pred} 衡量观测价值，同时将单程移动时间扩展为预计完成时间，使候选点评价同时计入前往测量点和测量后接近清除区域的移动成本。设当前位置和候选点分别为 M_{cur} 、 M ，新增观测后的预测圆心为 $c_{\text{MEC},\text{pred}}$ ，则

$$T_{\text{finish}}(M) = \frac{\|M_{\text{cur}} - M\|_2 + \mathbb{E}[\|M - c_{\text{MEC},\text{pred}}\|_2]}{5} + 6. \quad (3-4)$$

式 (3-4) 同时计入到达测量点和测量后接近清除区域的移动，并计入一次测向与频道切换的 6 s 成本。四项指标经极差归一化后，J+ 采用等权评分

$$J(M) = 0.25\tilde{Q}_{\text{geo}}(M) + 0.25\tilde{P}_{\text{rec}}(M) - 0.25\tilde{T}_{\text{finish}}(M) - 0.25\tilde{R}_{\text{pred}}(M). \quad (3-5)$$

令最高、最低得分为 J^* 和 J_- ，取 $\eta = 0.10$ ，构造 $\mathcal{R}_{\text{good}} = \{M: J(M) \geq J^* - \eta(J^* - J_-)\}$ 。在该集合中依次按预计完成时间最小、预测半径最小、综合得分最高选择下一检测点，从而减弱离散网格峰值对决策的影响。

(3) 覆盖点数与半径联合优化

未知环境覆盖路径规划通常需要在机器人运动过程中记录覆盖进度，并对尚未覆盖的局部区域安排补充访问，以降低覆盖空洞和远距离返程的风险[7]。结合本题作业区域为无障碍圆域、信号源存在接收距离下界的特点，本文不直接采用采样图覆盖算法，而是构造能够解析验证的“中心点—等角外环点”覆盖结构。模型设置一个原点停靠点和 n 个等角外环点，覆盖停靠点同时作为频道检测点，记为

$$\begin{aligned} M_0^{\text{cov}} &= (0,0), \quad M_\ell^{\text{cov}} = R_{\text{cov}} \left(\cos \frac{2\pi(\ell-1)}{n}, \sin \frac{2\pi(\ell-1)}{n} \right), \quad \ell \\ &= 1, \dots, n. \end{aligned} \quad (3-6)$$

原点覆盖内侧区域；外侧最不利位置位于作业圆边界及相邻环点角平分线上。该位置到最近外环点的距离以及给定 n 时的最小安全环半径分别为

$$d_n(R_{\text{cov}}) = \sqrt{R^2 + R_{\text{cov}}^2 - 2RR_{\text{cov}}\cos\frac{\pi}{n}},$$

$$R_{\text{cov},\min}(n) = R\cos\frac{\pi}{n} - \sqrt{1000^2 - R^2\sin^2\frac{\pi}{n}}. \quad (3-7)$$

只要 $d_n(R_{\text{cov}}) \leq 1000$ m，任意信号源均能在某个停靠点被发现。对应的开环覆盖路径长度为

$$L_{\text{cov}}(n, R_{\text{cov}}) = R_{\text{cov}} \left[1 + 2(n-1)\sin\frac{\pi}{n} \right]. \quad (3-8)$$

增加环点可以缩短安全半径和移动路径，但也会增加一轮未知频道扫描。综合比较 $n = 6, 7, 8$ 后，取 $n = 7$ 、 $R_{\text{cov}} = 1000$ m，此时 $d_7(1000) = 998.25$ m、 $L_{\text{cov}}(7, 1000) = 6207$ m，在最小接收半径下保留 1.75 m 覆盖余量。

(4) 共观测与滚动清除调度

机器狗为主频道到达某检测点后，同时评价其他已发现频道。设其保证接收比例和交会质量为 P_{rec} 、 q ，单位动作时间的信息价值为 $V_{\text{co}} = P_{\text{rec}}q/6$ 。当 $P_{\text{rec}} \geq 0.60$ 且 $V_{\text{co}} \geq 0.035$ 时执行共观测，每个覆盖点的预算为

$$B_{\text{co}} = \max \left\{ 5, \min \left(6, \left\lfloor \frac{N_{\text{unresolved}}}{3} \right\rfloor \right) \right\}. \quad (3-9)$$

共观测不增加移动距离；若某频道满足式 (3-3)，且绕行其圆心再前往下一覆盖点所增加的距离不超过 250 m，则顺路完成清除。

在每个停靠点，模型先扫描所有未知频道，再按 V_{co} 从高到低选择共观测对象。已满足清除条件、观测次数已充分或曾在该点测量的频道不再参与竞争。由此，覆盖阶段既保持未知频道的完整扫描，又利用停靠时间提前压缩已发现源的可行域。

覆盖结束后，令 $\mathcal{U}_t$ 为时刻 t 的未清除频道集合，以各频道最小覆盖圆圆心为锚点，求从当前位置出发的开放路径

$$\pi_t^* = \underset{\pi}{\operatorname{argmin}} [\| M_{\text{cur}} - c_{\text{MEC}, \pi(1)} \|_2 + \sum_{j=1}^{|\mathcal{U}_t|-1} \| c_{\text{MEC}, \pi(j)} - c_{\text{MEC}, \pi(j+1)} \|_2]. \quad (3-10)$$

分层导航方法可将较长时间尺度上的路径规划与环境状态变化触发的局部重规划分开处理，以增强动态任务中的适应能力[8]。借鉴其中“规划器—重规划触发器”的架构思想，本文以最近邻法生成初始开放路径，再采用 2-opt 和 Or-opt 改善任务顺序；但每轮只对 π_t^* 的首个频道执行一次测量或清除。动作完成后，将新的检测或清除响应视为重规划事件，更新全部频道状态并重新生成剩余路径。

具体而言，若首频道满足式 (3-3)，则前往其最小覆盖圆圆心清除，并在当前位置重新评价其他频道；若尚不能清除，则执行式 (3-5) 选出的检测点，获得新的方向或无信号信息。任一动作结束后均重建 $\mathcal{U}_t$ 和开放路径，因此一次观测产生的信息能够立即影响后续任务次序。

(5) 最终模型决策流程

综上，J+ 的决策过程包括以下五个环节：首先初始化 20 个频道状态并在原点完成首轮扫描；其次动态选择并访问七个外环点，对未知频道逐一检测；随后在各停靠点执行预算内共观测和低绕行顺路清除；覆盖结束后，以最小覆盖圆圆心构造开放路径，每次只执行首频道的一个动作；最后在每次响应后更新可行域、清除状态和任务顺序，直至全部信号源被可靠清除。模型由此形成“保证发现—保守定位—在线选点—滚动调度—可靠清除”的闭环。

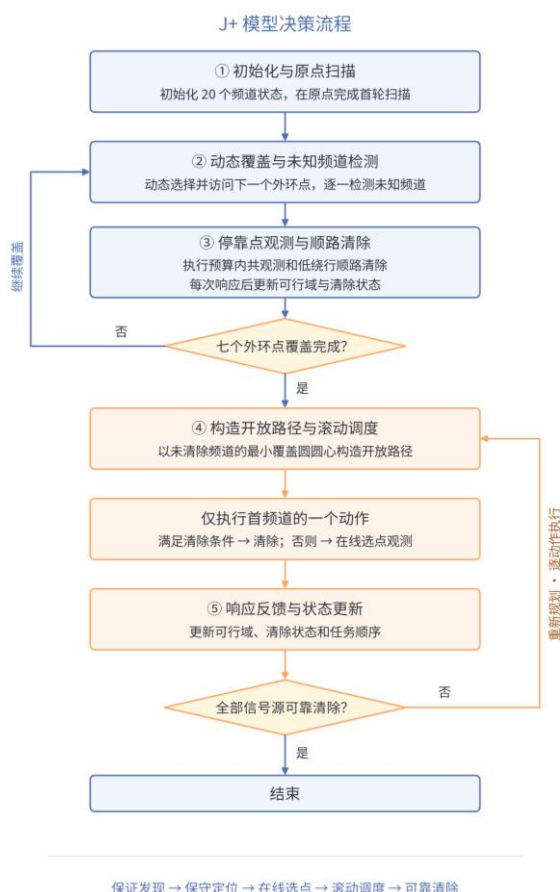


图 5-6J+模型的闭环决策流程

5.3.2 模型求解

(1) 评价指标与迭代准则

模型求解采用瓶颈驱动的迭代策略。为避免仅凭总耗时判断机制优劣，首先要求所有候选模型满足全清除约束，再以平均每源总耗时和 P95 每源总耗时为主目标，并用移动时间、检测与频道切换时间以及搜索后剩余定位负担解释收益来源。对于任一性能指标 ξ ，定义相邻模型的改善量为

$$\Delta\xi = \xi_{\text{旧}} - \xi_{\text{新}}. \quad (3-11)$$

由于主线模型的全清除率均为 100，每个信号源的成功清除时间恒为 5 s，故相邻模型平均每源总耗时的变化满足

$$\Delta T = \Delta T_{\text{move}} + \Delta T_{\text{detect+switch}} \quad (3-12)$$

式（3-12）用于核对每轮改进的时间收益是否闭合；发现至清除历时和搜索后剩余定位负担不参与加法核算，仅用于识别任务处理阶段的变化。整体求解沿“基础可行模型—选点成本优化—覆盖定位协同—滚动重规划—覆盖结构优化—测量调度修正”的顺序推进，其关系如图 5-7 所示。

第三问模型演进拓扑

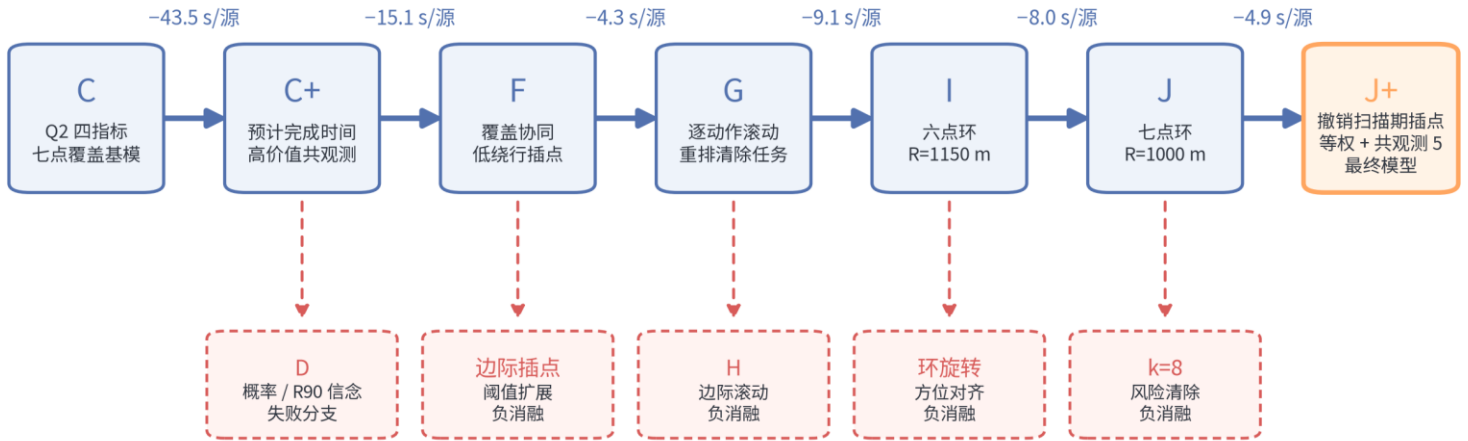


图 5-7 问题三模型演进及负消融路径

（2）选点成本与覆盖定位协同

主线试验表明，基础模型 C 已能实现全清除，但移动时间占比过高。C+ 将单程移动指标改为预计完成时间，并利用停靠点实施共观测，使平均每源耗时由 380.5 s 降至 337.0 s。随后，模型 F 将部分定位任务前移至覆盖阶段，使搜索后剩余定位负担由 82.7% 降至 48.7%，平均耗时进一步降至 321.9 s。

（3）滚动调度与覆盖结构优化

模型 G 将搜索后的逐源处理改为逐动作滚动重规划，每完成一次测量或清除便更新状态和任务顺序，平均耗时由 321.9 s 降至 317.6 s。在此基础上，模型 I 先将六点外环由 1250 m 收紧至 1150 m；模型 J 再联合比较外环点数与半径，最终选取 7 个半径为 1000 m 的外环点。

紧凑覆盖环下，扫描阶段主动插入测量点会造成额外绕行。最终模型 J+ 因而撤销扫描期插点，将主动测量并入滚动清除路径，同时采用四指标等权评分并将单停靠点共观测预算增至 5 次。相较 J，J+ 的移动与检测切换时间均下降，平均每源耗时由 300.5 s 降至 295.5 s，P95 由 380.8 s 降至 370.8 s。

（4）离线求解结果

在相同案例和随机数条件下进行 600 局配对试验，所得主线模型指标如表 5-4 所示。所有模型均实现 100 全清除，因而可以继续比较平均耗时及尾部耗时；发现至清除历时尚未按逐源口径重新统计，故不将原诊断量混入结果表。

表 5-4 问题三主线模型 600 局配对求解结果

指标	C	C+	F	G	I	J	J+
全清除率 (%)	100	100	100	100	100	100	100
平均总耗时 (s/源)	380.5	337.0	321.9	317.6	308.5	300.5	295.5
P95 总耗时 (s/源)	465.9	422.0	402.7	397.8	384.9	380.8	370.8
移动时间 (s/源)	310.4	265.3	263.5	258.8	249.6	236.8	235.4
检测与切换 (s/源)	65.2	66.7	53.5	53.8	53.9	58.7	55.1
搜索后剩余负担 (%)	82.7	82.7	48.7	48.7	48.6	42.6	46.6

除总体统计外，选取随机种子 24119 的第 231 个案例展示 J+ 的实际执行路线，如图 5-8 所示。该案例含 12 个真实信号源，用于直观说明覆盖搜索与滚动清除如何在同一条任务轨迹中衔接，不参与总体性能的统计推断。

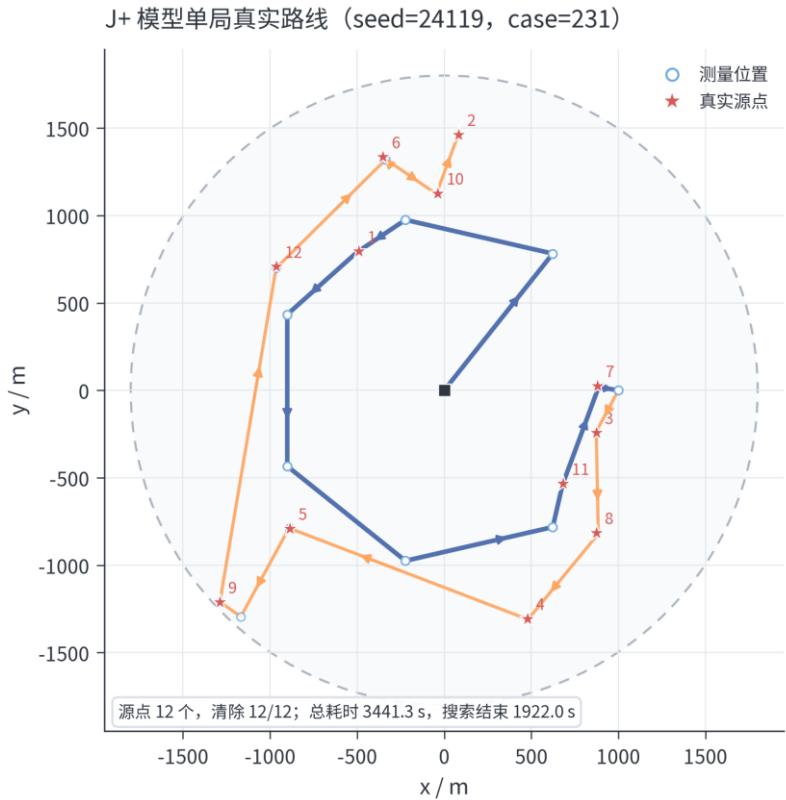


图 5-8 J+ 在代表性案例中的覆盖搜索与滚动清除路线图

图 5-8 中，黑色方块为出发原点，蓝色轨迹及空心圆反映紧凑覆盖骨架与测量停靠点，橙色轨迹表示覆盖完成后围绕已发现信号源展开的滚动定位与清除过程，红色星号为真实源点。机器人在覆盖阶段完成全域发现并积累多频道观测，随后将尚未完成的测量并入清除路径；最终 12 个信号源全部清除。图中阶段分离与前述“扫描期不插点、测量并入清除路径”的 J+ 调度规则一致。

综合表 5-4 与图 5-8 可见，模型从 C 迭代至 J+ 后，平均每源总耗时由 380.5 s 降至 295.5 s，累计减少 85.0 s，降幅为 22.3%；P95 由 465.9 s 降至 370.8 s，说明困难场景下

的尾部风险同步改善。J+ 的检测与切换时间并非全链条最低，搜索后剩余负担也不是最小，但其移动时间、平均总耗时与 P95 形成更优的综合结果，符合“先保证全清除，再优化均值和尾部”的词典序准则。因此，在题设基准生成条件下，选取 J+ 作为问题三的主要求解模型；在边界聚集的复合极端情景中，则保留模型 I 作为回退方案。

（5）正式测试结果与工程异常分析

在完成离线演练并冻结 J+ 的模型结构与参数后，使用官方模拟器进行了三次正式测试，结果见表 5-5。平均定位清除时间按虚拟总时间除以已清除干扰源个数计算，程序运行时间采用模拟器正式统计记录中的 `program_run_duration_ms`。

表 5-5 问题三三次正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间 (s/源)	程序运行时间 (s)
VQ68-NDNM-2CSA-8UAP	11	336.360	1.547
AHVB-CW98-XXVA-Q6MQ	13	274.998	1.338
RNZ5-BXVN-SY4H-22R4	14	269.432	1.499

正式测试 1 执行过程中，本地动作日志持久化模块发生 I/O 异常，异常处理程序主动调用 `/exit`，使任务提前结束，最终清除 11 个干扰源，该次测试不作为全清除案例。修复日志写入问题后，正式测试 2、3 分别实现 13/13 和 14/14 全清除，平均定位清除时间分别为 274.998 s/源和 269.432 s/源。三次测试均以 `user_exit` 结束，对应的加密行为日志已保持模拟器导出的原文件名并纳入支撑材料。该结果表明，J+ 的决策流程能够在正式接口中完成任务。

5.3.3 模型检验

为检验最终模型 J+ 的可靠性与适用边界，本文分别开展全域覆盖解析、三随机种子配对、负消融和七类压力情景试验。压力试验前冻结 I、J、J+ 的结构与参数，结果不再用于调参。

（1）全域覆盖解析检验

最终方案由原点和 7 个半径为 $R_{\text{cov}} = 1000 \text{ m}$ 的等角外环点构成。最不利位置位于作业圆边界与相邻环点角平分线的交点，将 $R = 1800 \text{ m}$ 、 $n = 7$ 代入式（3-7），得到

$$d_7(1000) = \sqrt{1800^2 + 1000^2 - 2 \times 1800 \times 1000 \cos \frac{\pi}{7}} = 998.25 \text{ m} \quad (3-13)$$

$$< 1000 \text{ m.}$$

因此，在接收半径取下界 1000 m 时，作业圆内任意信号源均至少位于一个停靠点的可靠接收范围内，最终覆盖结构满足全域发现要求。其最坏覆盖余量为 $1000 - 998.25 = 1.75 \text{ m}$ 。

在边界最坏扇区、最小接收半径及二者与 16 源满载、共享系统偏差叠加的三类直接覆盖压力情景中，I、J、J+ 共完成 5400 次运行，均未出现漏检或未发现案例，与式（3-13）的解析结论一致。

（2）三种子配对检验

采用随机种子 55021、24119 和 90317 生成 3000 个严格配对案例；同一案例仅改变模型，保持信号源位置、频道和观测扰动一致。设第 i 个案例中 J 与 J+ 的每源耗时差为 D_i ，其均值与 95% 置信区间按式（3-14）计算。

$$\bar{D} = \frac{1}{N} \sum_{i=1}^N D_i, \quad CI_{95\%} = \bar{D} \pm 1.96 \frac{s_D}{\sqrt{N}} \quad (3-14)$$

最终模型的大样本统计结果见表 5-6。

表 5-6 最终模型 J+ 的 3000 局离线仿真结果

指标	检验结果
全清除率	100%
平均耗时	293.5 s/源
中位数	289.2 s/源
P95 耗时	368.2 s/源
最大值	439.2 s/源
平均移动时间	233.7 s/源
平均检测与切换时间	54.8 s/源

3000 局均完成全部信号源清除。J+ 有 2063 局快于 J，平均节省 4.7 s/源，95% 置信区间为 [4.2, 5.2] s/源；相较 I 累计节省 12.5 s/源，置信区间为 [11.9, 13.1] s/源。两组区间均大于 0，说明最终改进并非由少数极端案例造成。

（3）机制与负消融检验

反向试验表明：八点外环的新增扫描成本超过路线缩短收益；将可靠清除半径由 20 m 放宽至 25 m 虽可缩短耗时，却会破坏全清除硬约束；覆盖环旋转、边际插入、近终态不抢占和继续增加共观测预算均未形成稳定净收益。因此，J+ 仅保留经同案例配对验证有效的机制。

（4）鲁棒性检验

三个独立种子下，J+ 相对 J 的改进方向一致，平均每源耗时极差约为 1.2 s，3000 局均实现全清除。进一步在基准分布及六类压力情景下，对 I、J、J+ 共运行 12600 次，未出现漏检、定位耗尽、失败清除或数值异常；各“情景—模型”组合 600 局零失败，对应单侧 95% 失败率上界约为 0.5%。

J+ 相对 J 在七类情景中均提速 5.1~7.6 s/源，95% 置信区间均大于 0，P95 改善 1.8~13.0 s/源。但在边界源集中于最坏扇区的 S6 复合情景中，模型 I 比 J+ 快 19.9 s/源。因此，J+ 适用于基准及 S0—S5 单项压力情景，极端边界聚集时回退至模型 I。

5.4 问题四的模型

5.4.1 模型建立

问题四含全向与定向干扰源。本问沿用第三问 J+模型的频道状态、可行域更新和清除框架，建立由认证覆盖、路线门控、保守定位和安全剪枝组成的 D 模型。仍以 X 表示信号源位置， M_i 表示第 i 个检测点， θ_i 表示示向度， $P_h^{\wedge}(k)$ 表示频道 h 经 k 次有效示向后的可行域。

(1) 定向传播条件下的模型修正

设频道 h 对应信号源的位置为 X ，其实际接收半径为 $r_h \in [1000, 1500] \text{ m}$ 。对于全向源，检测点 M 能够接收信号的充要距离条件为

$$\|M - X\|_2 \leq r_h. \quad (4-1)$$

对于定向源，除满足式 (4-1) 外，检测点还必须落在其前向 180° 发射半平面内。记 β_h 为定向源的发射中轴方向， $\alpha(X, M)$ 为从信号源 X 指向检测点 M 的方位角，并沿用前三问的环形角度归一化函数 $\text{wrap}(\cdot)$ ，则定向源的接收条件为

$$\|M - X\|_2 \leq r_h, \quad |\text{wrap}(\alpha(X, M) - \beta_h)| \leq 90^\circ. \quad (4-2)$$

检测成功且距离不小于近距离阈值时，接口返回从检测点指向信号源的示向度。设真实方位为 $\alpha(M_i, X)$ ，测向误差为 e_i ，则观测模型为

$$\theta_i = \alpha(M_i, X) + e_i, \quad |e_i| \leq \varepsilon = 1^\circ. \quad (4-3)$$

式 (4-2) 表明，无信号既可能由距离不足引起，也可能由波束背向引起，故不能据此直接删除位置区域；模型仅使用成功示向形成必然成立的约束。

沿用第三问的时间口径，设总移动距离为 L ，频道切换、测向、失败清除和成功清除次数分别为 N_{switch} 、 N_{measure} 、 $N_{\text{clear, fail}}$ 和 $N_{\text{clear, success}}$ ，机器狗速度为 5 m/s ，则总虚拟时间为

$$T = \frac{L}{5} + N_{\text{switch}} + 5N_{\text{measure}} + 3N_{\text{clear, fail}} + 5N_{\text{clear, success}}. \quad (4-4)$$

模型采用词典序目标：先保证全部真实信号源均被发现并可靠清除，再依次最小化平均每源总耗时和 P95 每源总耗时。

(2) 面向全域发现的认证覆盖模型

圆形距离覆盖不能排除测站全部落在定向源背面的情况。为此，本问构造 B、C 两组认证测站及访问路线，并同时检验接收距离与方位覆盖。对任意源位置 X ，先筛选与 X 距离不超过 1000 m 的测站，构成可达测站集合； 1000 m 为题设有效接收半径下界。记该集合的测站数为 $m(X)$ ，仅将 X 指向这些可达测站的方向角升序排列，并补充首角加 360° 形成循环序列。式 (4-5) 中的方向角均针对这一集合定义；当集合为空时，令最大循环角隙为 360° 。源位置与测站重合时可直接近距离检测并清除，单独处理。

$$G(X) = \max_{1 \leq j \leq m(X)} [\varphi_{j+1}(X) - \varphi_j(X)]. \quad (4-5)$$

若全部可达测站方向均能被某个开半圆容纳，则存在一种发射朝向使定向源对这些测站均背向；反之，若可达测站的最大循环角隙不超过 180° ，则任意前向半平面内至少包含一个距离不超过 1000 m 的测站，从而同时满足方向与接收距离条件。因此，认证路线应满足

$$\max_{X \in A_R} G(X) \leq 180^\circ. \quad (4-6)$$

程序在作业圆域内进行网格搜索与边界加密，各源位置仅使用 1000 m 内的可达测站计算角隙。B、C 路线分别含 27、29 个测站，最坏循环角隙为 162.46° 、 173.92° ，均小于 180° ，因而对任意源位和发射方向均至少存在一个可接收测站。

该保证仅用于未知频道的发现：每到认证测站均扫描尚未发现的频道，不对其实施距离剪枝；发现数未达上限 16 时必须完成所选认证路线。

(3) 面向场景适应的路线门控模型

B 路线在起源点无可见信号及边界朝外场景中具有较强稳健性，但路径较长；C 路线在一般随机场景中搜索日程更紧凑，并能在搜索阶段积累更多定位方位。为根据早期观测在二者之间选择，机器狗进入场地后先停留在原点 $M_0=(0,0)$ ，依次扫描 20 个频道，记发现的不同频道数为 n_0 。路线选择函数定义为

$$s(n_0) = \begin{cases} B, & n_0 = 0, \\ C, & n_0 \geq 1. \end{cases} \quad (4-7)$$

当 $n_0=0$ 时选择边界稳健的 B 路线，并为已发现频道保留 2 条有效示向；当 $n_0 \geq 1$ 时选择效率较高的 C 路线，将目标提高至 3 条示向，以减少搜索后的定位移动。

$$m_s = \begin{cases} 2, & s = B, \\ 3, & s = C. \end{cases} \quad (4-8)$$

原点是两条路线共有的首站，门控不增加移动，仅增加任务所需的频道检测，并根据早期可见性在发现稳健性与执行效率间切换。

(4) 面向执行效率的定位剪枝与可靠清除模型

对已发现频道 h ，第 i 次成功示向发生在检测点 M_i ，返回示向度 θ_i 。沿用前三问的半平面交方法，将示向误差带、1500 m 接收外包圆与作业圆域 A_R 相交，则 k 次有效示向后的保守可行域为

$$P_h^{(k)} = A_R \cap \bigcap_{i=1}^k (B_{i,h}^{\text{angle}} \cap B_{i,h}^{\text{range}}). \quad (4-9)$$

圆域、示向误差带和接收外包圆均为成功观测的必要条件，故真实源始终属于该可行域；定向源的无信号观测不参与可行域收缩。

搜索期到达新的认证检测点 M 后，模型只对已发现、未清除且有效示向数尚未达到 m_s 的频道考虑共观测。定义检测点到可行域的集合距离为

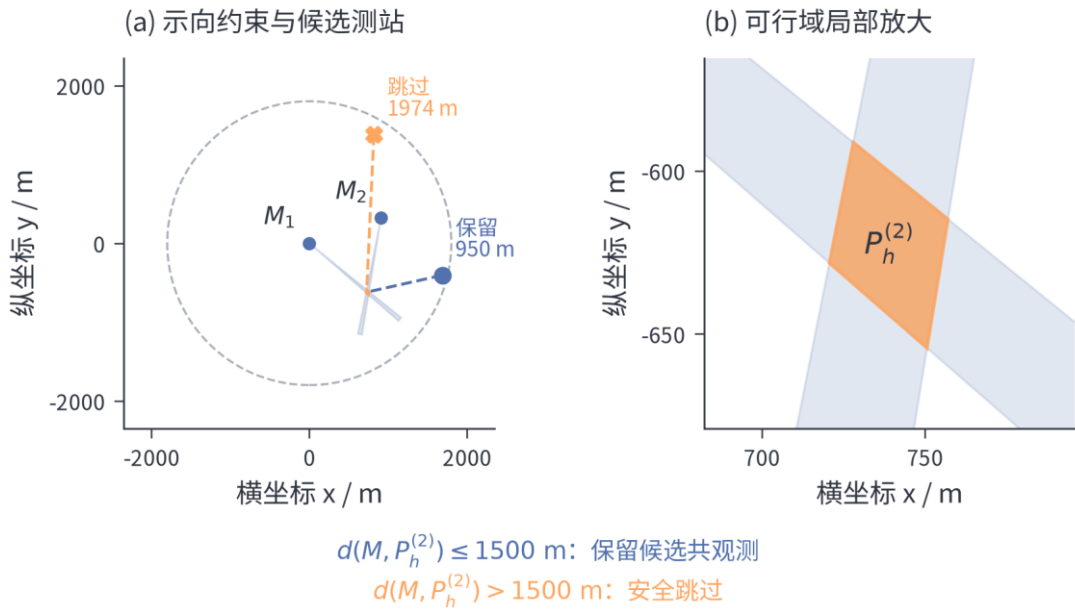
$$d(M, P_h^{(k)}) = \min_{Y \in P_h^{(k)}} \|M - Y\|_2. \quad (4-10)$$

当 M 位于凸多边形 $P_h^{(k)}$ 内时，该距离为 0；否则逐边计算点到线段的最小距离。由此建立安全距离门：

$$d(M, P_h^{(k)}) > 1500 \text{ m} \Rightarrow \text{skip}(M, h) = 1. \quad (4-11)$$

由于保守可行域包含真实源，检测点到真实源的距离不小于其到可行域的最小距离。式 (4-11) 成立时，该次测量必然失败。因此，距离门只删除无效共观测，不改变未知频道的认证发现过程。

图 5-9 利用正式日志中的两次成功示向重建可行域，并以路线测站展示距离门：最短距离约 950 m 时保留候选共观测，约 1974 m 时安全跳过。候选站点用于解释判据，并非实际剪枝记录；保留也不意味着必然接收到信号。



真实示向重建；候选站点为判据演示，并非实际剪枝记录。
蓝色为 $\pm 1^\circ$ 示向约束；橙色为保守可行域；保留不代表必然接收。

图 5-9 基于保守可行域的共观测距离剪枝机制

保留的有效示向继续按式 (4-9) 更新可行域，并按第三问判据实施可靠清除。若多方位交会退化，则以最后一次示向为中轴，在 $0 \sim 1500 \text{ m}$ 内布置交错点阵：纵向间隔 30 m ，横向偏移为 $\pm 10 \text{ m}$ 和 $\pm 30 \text{ m}$ 。任意位置到最近清除点的最大距离满足

$$d_{\text{gap}} = \sqrt{15^2 + 10^2} \approx 18.03 \text{ m} < 20 \text{ m}. \quad (4-12)$$

示向误差造成的最大横向偏差为 $1500 \sin 1^\circ \approx 26.18 \text{ m}$ ，落在 $\pm 30 \text{ m}$ 点阵宽度内，因此该点阵可用 20 m 清除半径覆盖完整误差走廊。

5.4.2 模型求解

(1) 认证测站与访问路线的确定

认证路线先进行测站设计，再优化访问顺序。候选测站必须满足式（4-6）的最大循环角隙约束；以原点为固定起点，将保留测站组织为开放路径。设路线 s 的测站集合为 M_s 、访问排列为 π_s ，其目标为

$$\pi_s^* = \arg \min_{\pi_s} \sum_{j=1}^{|\mathcal{M}_s|-1} \|M_{\pi_s(j+1)}^{\text{cov}} - M_{\pi_s(j)}^{\text{cov}}\|_2. \tag{4-13}$$

采用多起点最近邻生成初始路线，再以 2-opt 和 Or-opt 消除交叉与折返。候选路线除比较长度外，还通过离线仿真比较发现日程、搜索后移动和平均每源耗时，最终冻结 B、C 路线，参数见表 5-7。

表 5-7 B、C 认证路线主要参数

路线	测站数	开放路线长度/m	最坏循环角隙	搜索期目标方位数	主要适用场景
B	27	20409.36	162.46°	2	原点零发现及边界困难场景
C	29	18147.63	173.92°	3	原点有发现的一般场景

C 路线比 B 多 2 个测站，但开放路线缩短 2261.73 m，适合原点已有发现的一般场景；B 的最坏循环角隙更小，适合边界朝外场景。两条路线均满足式（4-6），在线阶段仅在二者之间门控。

图 5-10 给出了冻结的 B、C 测站布局及访问方向。虚线圆表示干扰源分布区域，星形与方形分别表示起点和终点；检测点可位于目标区域外，路线均为不返回起点的开放路径。

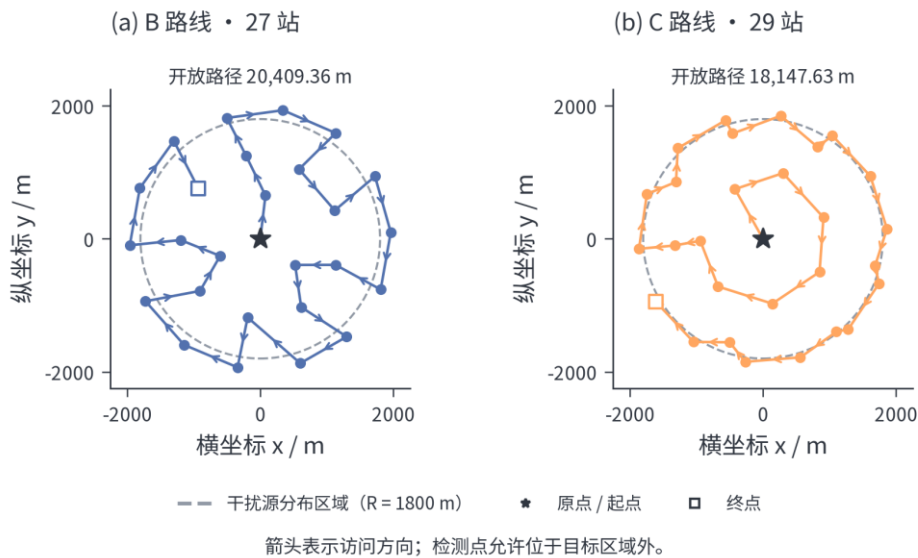


图 5-10 B、C 认证测站布局与开放访问路线

（2）模型迭代与参数筛选

模型采用瓶颈驱动迭代：候选版本先满足全清除约束，再在同案例条件下比较平均每源耗时、P95 及移动、检测和切换耗时。B、C 提供两类认证路线；C+ 引入原点门控；随后增加搜索期第三方位和保守距离过滤，并将距离门扩展到 B、C 两个分支，形成 D 模型。

门控阈值在 0、1、2、3 中同案例筛选，阈值继续提高未获稳定收益，故取 0，即“零发现走 B，否则走 C”。C 分支将搜索期目标方位数由 2 提高至 3，并以式（4-11）的保守距离门删除必然失败的共观测。

距离门扩展到 B 分支后，一般场景与 C+3F 逐例一致，两类边界场景的检测和切换耗时下降，最终版本记为 D。相较第三问，D 保留滚动定位与清除框架，新增定向可见性约束、认证路线门控及安全距离剪枝，直接针对定向源漏检风险；动态访问顺序和延迟门控因缺乏跨场景稳定优势而舍弃。

（3）正式测试求解结果

在官方模拟器中完成三组正式测试，结果见表 5-8。三组案例均清除全部信号源，且无无效操作或越界拒绝，说明 D 模型能够在正式接口中完成搜索、定位与清除闭环。

表 5-8 问题四正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间 (s/源)	程序运行时间 (s)
CUE7-UQXY-C2CP-EDAR	12	549.678	4.564
QKWP-TYM6-ERGS-TD3J	16	285.047	1.926
WN66-ZTSM-A7JF-XYH4	13	532.680	3.784

注：总虚拟时间包括移动、检测、清除及状态切换时间；程序时间不计入题设虚拟时间。

三次正式测试的原点发现数分别为 8、5、9，均进入 C 分支。因此，正式结果验证了模型的端到端可执行性和 C 分支效果；B 分支则由后文的离线配对与鲁棒性试验检验。

5.4.3 模型检验

（1）定向发现覆盖检验

在作业圆域内对源位进行网格搜索，并计算其指向距离不超过 1000 m 的可达认证测站的循环最大角隙。B、C 路线的最坏值分别为 162.46° 和 173.92°，均小于临界角 180°，故任意源位和发射方向至少对应一个前向测站。

（2）安全剪枝正确性检验

理论上，保守可行域始终包含真实源；当检测点到该区域的最小距离超过 1500 m 时，被删共观测必然失败。三类场景共剪除 34473 次检测，其中真实可接收次数为 0，验证了剪枝不影响未知频道扫描或有效示向获取。

（3）同案例配对性能检验

在相同源位、传播方式和测向扰动下，对 D 与 C+3F 进行 450 组配对。三类场景均全部清除；普通场景逐例一致，边界随机与边界朝外场景的平均耗时分别降低 3.61% 和 2.92%，且配对差的 95% 置信区间均位于 0 以下，说明距离剪枝在不降低全清除率的前提下稳定提速。

(4) 源数量分层与鲁棒性检验

将源数量扩展为 10~16 的全部整数，并在普通、边界随机和边界朝外三类场景下分层复测。各层均全部清除，D 的平均耗时及相对 C+3F 的改善见表 5-9。

表 5-9 不同源数量下 D 模型的鲁棒性检验

源数量	普通场景	边界随机	边界朝外	D-C+3F（随机/朝外）
10	745.3	886.4	889.4	-19.11/-13.67
11	674.9	799.5	808.1	-13.53/-15.74
12	620.6	749.5	746.3	-14.42/-17.06
13	566.4	696.4	697.3	-20.67/-14.77
14	518.3	629.6	625.3	-13.46/-8.56
15	493.4	606.0	599.4	-17.63/-14.11
16	405.5	569.3	531.8	-21.98/-14.23

共构造 1210 个独立源场景并完成 6250 次模型运行，其中题设内 6150 次均全部清除；表 5-9 的 14 组边界配对差均为负，且 95% 置信区间上界均小于 0。按零失败的“三倍法则”，测试分布下单次运行失败率的 95% 上界约为 0.05%，说明模型在题设范围内具有稳定的全清除能力。

(5) 适用边界分析

结论以测向误差不超过 $\pm 1^\circ$ 、距离剪枝仅作用于搜索期已发现频道为前提。正式测试中距原点 1868.945 m 的测站可正常执行，专项探测还验证至 1966 m，表明 1800 m 是信号源布设范围而非机器人硬活动边界。九类空间对抗场景均实现全清除，但原点门控仍存在效率遗憾，其中最小半径朝内场景的平均遗憾为 165.35 s/源；因此门控不是全局最优，只能视为在缺少先验信息时的稳健折中。

6 模型评价与改进

6.1 模型优点

- (1) 清除率有硬保证。闭半平面交、最小覆盖圆和覆盖骨架分别保证定位、清除和全域发现；3000 局配对试验与 12600 次压力运行清除率均为 100%。
- (2) 模型实现具有可行性与可移植性。Q2 主动选点、Q3 多源协同和 Q4 定向源处理均已模块化实现，并经多轮本地运行检验；各模块替换场景参数即可复用，便于迁移至不同任务环境。统一接口可部署于不同 Python 运行环境。

(3) 结构清晰且易扩展，参数可按场景快速重标定。

6.2 模型局限

(1) 传播条件较理想化，未显式考虑遮挡、多径和短时场强波动。

(2) 部分参数依赖模拟场景。极端边界聚集时模型 I 比 J+ 快 19.9 s/源，说明固定覆盖结构并非始终最优。

6.3 模型改进

(1) 细化传播建模。引入距离—方位联合接收概率模型，并用贝叶斯更新在线修正信念区域。

(2) 挖掘速度余量。以多情景鲁棒优化替代单一分布标定，依据早期观测动态选择覆盖半径、环点数和调度参数。

7 AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于赛题理解与问题分析、符号梳理、算法设计辅助。

同时，AI 工具还用于求解代码编写与调试、离线仿真与鲁棒性试验脚本生成、统计结果整理，以及部分图表说明。

AI 生成的建模建议、代码及文字均经队员结合题意修改。模型结构、参数设置、结果分析及核心结论均由本队主导完成，详细使用情况见支撑材料。

8 参考文献

- [1] TOKEKAR P, VANDER HOOK J, ISLER V. Active target localization for bearing based robotic telemetry[C]//2011 IEEE/RSJ International Conference on Intelligent Robots and Systems. San Francisco, CA, USA: IEEE, 2011. DOI: 10.1109/IROS.2011.6095177.
- [2] ZHAO S, CHEN B M, LEE T H. Optimal placement of bearing-only sensors for target localization[C]//Proceedings of the 2012 American Control Conference. Montreal, QC, Canada: IEEE, 2012: 5108-5113. DOI: 10.1109/ACC.2012.6314884.
- [3] 罗双喜. 多站交叉定位相对 GDOP 及其测向站分布问题研究[J]. 指挥控制与仿真, 2020, 42(2): 7-11. DOI: 10.3969/j.issn.1673-3819.2020.02.002.
- [4] VANDER HOOK J, TOKEKAR P, ISLER V. Cautious greedy strategy for bearing-only active localization: analysis and field experiments[J]. Journal of Field Robotics, 2014, 31(2): 296-318. DOI: 10.1002/rob.21499.
- [5] VANDER HOOK J, TOKEKAR P, BRANSON E, et al. Local-search strategy for active localization of multiple invasive fish[C]//DESAI J P, DUDEK G, KHATIB O, et al. Experimental Robotics: The 13th International Symposium on Experimental Robotics. Heidelberg: Springer, 2013: 859-873. DOI: 10.1007/978-3-319-00065-7_57.

- [6] 姚胤博, 许睿, 徐悦, 等. 卫星导航干扰源无人机定位观测路径规划方法[J]. 全球定位系统, 2024, 49(6): 72-83. DOI: 10.12265/j.gnss.2024097.
- [7] SHEN Z, WILSON J P, GUPTA S. C*: A coverage path planning algorithm for unknown environments using rapidly covering graphs[J]. IEEE Transactions on Robotics, 2026, 42: 1233-1253. DOI: 10.1109/TRO.2026.3661719.
- [8] LIU S, ZHANG Y, LI G, et al. A hierarchical adaptive navigation planner for UAVs in 3D complex environments based on deep reinforcement learning[J]. Applied Soft Computing, 2026, 190: 114614. DOI: 10.1016/j.asoc.2026.114614.

9 附录

附录 A 支撑材料文件清单

```
q1_exact_verify.py
q1_solver.py
q2_actual_replay.py
q2_solver.py
q2_weight_calibrate.py
q3_coobserve.py
q3_coverage_ring.py
q3_feasible_region.py
q3_scheduler.py
q3_selector.py
q4_exact_verify.py
q4_solver.py
q1_experiment_data.json
q2_candidate_scores.csv
q2_formal_strategy_comparison.csv
q2_formal_weight_robustness.csv
q2_weight_grid_calibration.csv
q3_jplus_final_actions.json
q3_jplus_robustness_seed24119_summary.json
q3_jplus_robustness_seed55021_summary.json
q4_b_route_design.json
q4_c_route_design.json
q4_robustness_report.json
q3_formal_test_formal-p3-1-VQ68-NDNM-2CSA-8UAP.jlog
q3_formal_test_formal-p3-2-AHVB-CW98-XXVA-Q6MQ.jlog
q3_formal_test_formal-p3-3-RNZ5-BXVN-SY4H-22R4.jlog
q4_formal_test_formal-p4-1-CUE7-UQXY-C2CP-EDAR.jlog
q4_formal_test_formal-p4-2-QKWP-TYM6-ERGS-TD3J.jlog
q4_formal_test_formal-p4-3-WN66-ZTSM-A7JF-XYH4.jlog
```

附录 B 完整源程序

q1_exact_verify.py

以旋转卡壳与全顶点枚举精确求区域直径，并判定以直径 D 为直径的同直径圆能否覆盖区域。

```
from __future__ import annotations

from fractions import Fraction
from itertools import combinations
import math

Point = tuple[Fraction, Fraction]
```

```

def _cross(a: Point, b: Point, c: Point) -> Fraction:
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])

def _distance2(a: Point, b: Point) -> Fraction:
    return (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2

def convex_hull(points) -> tuple[Point, ...]:
    ordered = sorted({(Fraction(p[0]), Fraction(p[1])) for p in points})
    if len(ordered) <= 1:
        return tuple(ordered)
    lower, upper = [], []
    for part, sequence in ((lower, ordered), (upper, reversed(ordered))):
        for p in sequence:
            while len(part) >= 2 and _cross(part[-2], part[-1], p) <= 0:
                part.pop()
            part.append(p)
    return tuple(lower[:-1] + upper[:-1])

def convex_diameter(vertices, method: str = "calipers"):
    """第三步：区域直径  $D = \max_{\{i < j\}} ||v_i - v_j||$ ，同时返回最远点对。

    任一内部点的凸组合由三角不等式不会超过顶点对距离，故只需检查顶点；
    距离平方保留有理数精确比较，最后才开平方。复杂度：枚举  $O(m^2)$ ，卡壳  $O(m)$ 。
    """
    if method not in ("calipers", "exhaustive"):
        raise ValueError("method 必须为 calipers 或 exhaustive")
    hull = convex_hull(vertices)
    if not hull:
        raise ValueError("空点集没有可用直径")
    best = Fraction(0)
    endpoints = (hull[0], hull[0])

    def consider(a: Point, b: Point) -> None:
        nonlocal best, endpoints
        squared = _distance2(a, b)
        if squared > best:
            best, endpoints = squared, (a, b)

    size = len(hull)
    if method == "exhaustive" or size <= 2:
        for a, b in combinations(hull, 2):
            consider(a, b)
    else:
        j = 1
        for i in range(size):
            nxt = (i + 1) % size

            def area(index: int) -> Fraction:
                return _cross(hull[i], hull[nxt], hull[index % size])

            while area(j + 1) > area(j):
                j = (j + 1) % size
            consider(hull[i], hull[j])
            consider(hull[nxt], hull[j])
            if area(j + 1) == area(j):
                # 相邻对侧顶点面积相等（平行支撑边）时同时检查候选点，避免漏检。
                consider(hull[i], hull[(j + 1) % size])
                consider(hull[nxt], hull[(j + 1) % size])
    return math.sqrt(best), best, endpoints

def diameter_circle_coverage(vertices):
    """第四步：是否存在直径等于区域直径的圆覆盖区域。

```

半径 $D/2$ 的圆若要同时包含最远点对 a, b , 圆心只能是中点 $c=(a+b)/2$;
 圆盘为凸集, 故只检查 $\max_i ||v_i - c|| \leq D/2$ 即可, 结论精确 (不用浮点容差)。
 等边三角形反例中 $R_{\min} = D/\sqrt{3} > D/2$, 判定为「否」。

```

"""
hull = convex_hull(vertices)
_, squared_diameter, (a, b) = convex_diameter(hull)
center = ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
farthest = max(hull, key=lambda p: _distance2(p, center))
maximum = _distance2(farthest, center)
radius2 = squared_diameter / 4
radius = math.sqrt(radius2)
covers = maximum <= radius2
# 正值为最远顶点超出圆边界的量, 负值表示仍有余量, 仅供显示。
return center, radius, covers, math.sqrt(maximum) - radius, farthest

```

q1_solver.py

把每条 $\pm 1^\circ$ 示向度误差带转成闭半平面并逐次求交, 得到凸定位区域并判定空集、无界与退化。

```

from __future__ import annotations

from fractions import Fraction
from itertools import combinations
import math
from typing import Iterable, Literal, Sequence

Number = int | float | Fraction
Point = tuple[Fraction, Fraction]
RegionStatus = Literal["empty", "unbounded", "point", "segment", "polygon"]

def _number(value: Number) -> Fraction:
    if isinstance(value, float) and not math.isfinite(value):
        raise ValueError("坐标、角度和系数必须是有限数值")
    return Fraction(value)

def _cross(a: Point, b: Point, c: Point) -> Fraction:
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])

def _direction_degrees(angle: Fraction) -> Point:
    """按八分圆计算方向, 保持轴向/对角向精确以及反向、旋转对称。

    向量不必归一化。普通角度仍受浮点三角函数精度限制; 这里不放宽误差界。
    """
    quadrant, remainder = divmod(angle % 360, 90)
    swap = remainder > 45
    reduced = 90 - remainder if swap else remainder
    if reduced == 0:
        x, y = Fraction(1), Fraction(0)
    elif reduced == 45:
        x, y = Fraction(1), Fraction(1)
    else:
        radians = math.radians(float(reduced))
        x, y = _number(math.cos(radians)), _number(math.sin(radians))
    if swap:
        x, y = y, x
    return ((x, y), (-y, x), (-x, -y), (y, -x))[int(quadrant)]

def bearing_halfplanes(

```

```

stations: Iterable[Sequence[Number]],
bearings_deg: Iterable[Number],
error_deg: Number = 1.0,
) -> tuple[tuple[Fraction, Fraction, Fraction], ...]:
    """第一步：每条观测的楔形  $W_i = \{X \mid \text{cross}(u^-, X-M) \geq 0 \text{ 且 } \text{cross}(u^+, X-M) \leq 0\}$ 。

    等价于两个闭半平面  $a \cdot x + b \cdot y \leq c$ ，共  $2n$  个约束；方位从正东逆时针计量，
    允许负角度或超过  $360^\circ$ ； $0 < \text{error\_deg} < 90$  保证张角小于  $180^\circ$ 。
    """
    points = tuple((_number(x), _number(y)) for x, y in stations)
    angles = tuple(_number(a) for a in bearings_deg)
    epsilon = _number(error_deg)
    if len(points) != len(angles):
        raise ValueError("检测点数量必须与示向度数量相同")
    if not 0 < epsilon < 90:
        raise ValueError("误差半角必须满足  $0 < \text{error\_deg} < 90$ ")
    result = []
    for (x, y), angle in zip(points, angles):
        lower = _direction_degrees(angle - epsilon)
        upper = _direction_degrees(angle + epsilon)
        if lower[0] * upper[1] - lower[1] * upper[0] <= 0:
            raise ValueError("误差半角接近浮点分辨率极限，边界方向无法可靠区分")
        for a, b in (
            (lower[1], -lower[0]),
            (-upper[1], upper[0]),
        ):
            a, b = _number(a), _number(b)
            result.append((a, b, a * x + b * y))
    return tuple(result)

def convex_hull(points: Iterable[Sequence[Number]]) -> tuple[Point, ...]:
    """二维单调链凸包：逆时针、去重、删除共线中间点，不重复首点。"""
    ordered = sorted({_number(p[0]), _number(p[1])} for p in points)
    if len(ordered) <= 1:
        return tuple(ordered)
    lower, upper = [], []
    for part, sequence in ((lower, ordered), (upper, reversed(ordered))):
        for p in sequence:
            while len(part) >= 2 and _cross(part[-2], part[-1], p) <= 0:
                part.pop()
            part.append(p)
    return tuple(lower[:-1] + upper[:-1])

def intersect_halfplanes(
    halfplanes: Iterable[tuple[Number, Number, Number]],
) -> tuple[RegionStatus, tuple[Point, ...], Point | None, Point | None]:
    """第二步：通用闭半平面交，无人工包围框，支持退化和无界。

    枚举边界交点并按全部约束筛选， $O(k^3)$  次有理数运算，适合少量测向观测。
    返回（区域状态，顶点，可行点，无界延伸方向）。
    """
    constraints = tuple((_number(a), _number(b), _number(c)) for a, b, c in halfplanes)
    if any(a == b == 0 and c < 0 for a, b, c in constraints):
        return "empty", (), None, None
    constraints = tuple((a, b, c) for a, b, c in constraints if a or b)

    def feasible(p: Point) -> bool:
        return all(a * p[0] + b * p[1] <= c for a, b, c in constraints)

    origin = (Fraction(0), Fraction(0))
    witness = origin if feasible(origin) else None
    # 非空闭凸集距原点最近点必为原点、边界垂足或边界交点之一。
    if witness is None:

```

```

        for a, b, c in constraints:
            norm2 = a * a + b * b
            p = (a * c / norm2, b * c / norm2)
            if feasible(p):
                witness = p
                break
    vertices = set()
    for (a1, b1, c1), (a2, b2, c2) in combinations(constraints, 2):
        determinant = a1 * b2 - a2 * b1
        if determinant == 0:
            continue
        p = ((c1 * b2 - c2 * b1) / determinant, (a1 * c2 - a2 * c1) / determinant)
        if feasible(p):
            vertices.add(p)
            witness = p
    if witness is None:
        return "empty", (), None, None

    # 无界等价于存在  $d \neq 0$ , 使全部法向量满足  $n \cdot d \leq 0$ ;
    # 二维非平凡衰退锥必包含某条约束边界的切向方向。
    directions = [(Fraction(1), Fraction(0))] if not constraints else []
    for a, b, _ in constraints:
        directions.extend([(b, -a), (-b, a)])
    for d in directions:
        if all(a * d[0] + b * d[1] <= 0 for a, b, _ in constraints):
            return "unbounded", (), witness, d
    hull = convex_hull(vertices)
    if not hull:
        raise ArithmeticError("非空有界区域未找到顶点")
    status: RegionStatus = "point" if len(hull) == 1 else "segment" if len(hull)
    == 2 else "polygon"
    return status, hull, witness, None

def localize(stations, bearings_deg, error_deg: Number = 1.0):
    """串联两步: 观测 -> 半平面 -> 定位区域 (空集与无界不返回直径)。"""
    region = intersect_halfplanes(bearing_halfplanes(stations, bearings_deg,
    error_deg))
    return region

```

q2_actual_replay.py

按真实接收半径和第二次角度误差回放检测结果, 并以 MEC 判断能否清除。

```

from __future__ import annotations

import math
import sys
from pathlib import Path

Q2_DIR = Path(__file__).resolve().parents[1] / "q2"
sys.path.insert(0, str(Q2_DIR))
from q1_q2_bridge import bearing, evaluate_a2 # noqa: E402

S1 = (-750.0, -450.0)
FAILURE_MEC = 250.0

def wrap(angle_deg: float) -> float:

```

```

        return (angle_deg + 180.0) % 360.0 - 180.0

    def replay_second_measurement(g_true: tuple[float, float], receive_radius_m: float,
                                  first_bearing_deg: float, second_error_deg: float,
                                  s2: tuple[float, float]) -> dict:
        """未接收时不能假装定位成功；接收时才以 A2 的实际 MEC 评价清除条件。"""
        distance = math.dist(s2, g_true)
        if distance > receive_radius_m:
            return {"return_type": "no_signal", "received": False, "mec_m": FAILURE_MEC,
                    "can_clear": False}
        if distance <= 5.0:
            return {"return_type": "near", "received": True, "mec_m": 0.0,
                    "can_clear": True}
        second_bearing = wrap(bearing(s2, g_true) + second_error_deg)
        region = evaluate_a2(S1, first_bearing_deg, s2, second_bearing)
        mec = region.mec_radius_m
        return {"return_type": "direction", "received": True, "mec_m": mec,
                "can_clear": math.isfinite(mec) and mec <= 20.0}

```

q2_solver.py

以四指标评分完成“全局粗搜索—局部加密—高分区执行点”选择

```

from __future__ import annotations

import math
from typing import Callable

S1 = (-750.0, -450.0)
ETA = 0.10

def normalize_and_score(rows: list[dict], weights: tuple[float, float, float, float]) -> list[dict]:
    """对 q_geo、p_rec、travel_s、r_pred_m 分别归一化后计算 J。"""
    valid = [r for r in rows if r["p_rec"] > 0 and math.isfinite(r["r_pred_m"])]
    if not valid:
        raise RuntimeError("没有保证接收的候选点")
    for source, target in (("q_geo", "q_n"), ("p_rec", "p_n"),
                           ("travel_s", "t_n"), ("r_pred_m", "r_n")):
        lo, hi = min(r[source] for r in valid), max(r[source] for r in valid)
        for row in rows:
            row[target] = (row[source] - lo) / (hi - lo) if row in valid and hi > lo else 0.0
    for row in rows:
        if row not in valid:
            row["J"] = -math.inf
        else:
            # +w3+w4 是不改变排序的常数平移，使 J>=0 后可直接采用 90% 阈值。
            row["J"] = (weights[0] * row["q_n"] + weights[1] * row["p_n"]
                        - weights[2] * row["t_n"] - weights[3] * row["r_n"]
                        + weights[2] + weights[3])
    return rows

def polar_point(bearing_deg: float, distance_m: float, offset_deg: float) -> tuple[float, float]:
    angle = math.radians(bearing_deg + offset_deg)
    return S1[0] + distance_m * math.cos(angle), S1[1] + distance_m *

```

```

math.sin(angle)

def choose_second_point(bearing_deg: float, weights: tuple[float, float, float,
float],
                        metric_fn: Callable[[tuple[float, float]], dict]) ->
tuple[dict, list[dict]]:
    """metric_fn 必须返回 q_geo、p_rec、travel_s、r_pred_m 四项原始指标。"""
    coarse_points = [polar_point(bearing_deg, d, offset)
                      for d in range(100, 1801, 200) for offset in range(0, 360,
30)]
    coarse = [{"x_m": x, "y_m": y, **metric_fn((x, y))} for x, y in coarse_points]
    normalize_and_score(coarse, weights)
    seeds = sorted(coarse, key=lambda r: r["J"], reverse=True)[:2]

    refined_points: set[tuple[float, float]] = set()
    for seed in seeds:
        distance = math.dist(S1, (seed["x_m"], seed["y_m"]))
        direction = math.degrees(math.atan2(seed["y_m"] - S1[1], seed["x_m"] -
S1[0]))
        for d in (max(50.0, distance - 100.0), distance, min(1800.0, distance +
100.0)):
            for offset in (-10.0, 0.0, 10.0):
                refined_points.add(tuple(round(v, 7) for v in polar_point(0.0, d,
direction + offset)))
    refined = [{"x_m": x, "y_m": y, **metric_fn((x, y))} for x, y in
refined_points]
    normalize_and_score(refined, weights)

    peak = max(refined, key=lambda r: r["J"])
    good = [r for r in refined if r["J"] >= (1.0 - ETA) * peak["J"]]
    # 高分区内优先短移动; 若相同, 再选预测 MEC 更小的点。
    execute = min(good, key=lambda r: (round(r["travel_s"], 6), r["r_pred_m"], -
r["J"]))
    return execute, good

```

q2_weight_calibrate.py

以蒙特卡洛场景和单纯形网格校准四指标权重, 并进行归一化扰动检验

```

from __future__ import annotations

def simplex_grid(step: float = 0.25) -> list[tuple[float, float, float, float]]:
    """列举 w_i>=0、sum(w_i)=1 的四维权重网格。"""
    n = round(1 / step)
    return [(a / n, b / n, c / n, (n - a - b - c) / n)
            for a in range(n + 1)
            for b in range(n - a + 1)
            for c in range(n - a - b + 1)]

def outer_loss(mean_mec_m: float, receive_rate: float, mean_time_s: float,
baseline_time_s: float) -> float:
    """以实际二测 MEC、接收率和总时间构造外层损失, 越小越好。"""
    return 0.40 * mean_mec_m / 20.0 + 0.35 * (1.0 - receive_rate) + 0.25 *
mean_time_s / baseline_time_s

def normalize_perturbation(weights: tuple[float, float, float, float], index: int,
factor: float) -> tuple[float, float, float, float]:
    """只扰动一个权重后重新归一化, 保持总权重恒为 1。"""
    changed = list(weights)
    changed[index] *= factor
    total = sum(changed)

```

```

        return tuple(value / total for value in changed)

def calibrate(weights_grid, scenarios, evaluate_policy, baseline_time_s: float):
    """evaluate_policy(w, scenarios) 应返回实际回放的 MEC、接收率和时间均值。"""
    result = []
    for weights in weights_grid:
        metrics = evaluate_policy(weights, scenarios)
        loss = outer_loss(metrics["mean_mec_m"], metrics["receive_rate"],
metrics["mean_time_s"], baseline_time_s)
        result.append({"weights": weights, "loss": loss, **metrics})
    return sorted(result, key=lambda row: row["loss"])

```

q3_coobserve.py

在主动测量停靠点对其它已发现未清除频道做"顺便测量", 按单位时间信息价值筛选前几个。

```

from __future__ import annotations

import math

MIN_RECEIVE_R = 1000.0      # 保证接收半径下界
MIN_P_REC = 0.60           # 参与共观测的接收概率下限
VALUE_PER_SECOND_MIN = 0.035 # 单位时间价值下限 (一次测量 6 s)
MAX_COOBSERVATIONS_CPLUS = 3 # C+~J: 环点每站最多 3 个
MAX_COOBSERVATIONS_JPLUS = 5 # J+: 预算下限提高到 5 (原点仍为 3)
MEASURE_SECONDS = 6.0

def wrap(angle_deg):
    return (angle_deg + 180.0) % 360.0 - 180.0

def bearing(a, b):
    return math.degrees(math.atan2(b[1] - a[1], b[0] - a[0]))

def
coobserve(agent, point, primary_channel,
max_coobservations=MAX_COOBSERVATIONS_CPLUS):
    """已到达 point 测完主频道后, 挑选值得"顺便测量"的其它频道并按价值降序执行。

    不增加任何移动, 只花 6 s 测量时间; 因此判据是单位时间信息价值
    value = p_rec × 交会质量 / 6 s,
    其中 p_rec 为候选点保证接收的样本占比, 交会质量为与历史各站交角  $\sin^2$  的最大值均值。
    """
    choices = []
    for channel, st in agent.state.items():
        if channel == primary_channel or st.cleared or st.exhausted or not
st.discovered:
            continue
        if len(st.observations) >= 5 or any(math.dist(point, old) <= 1.0 for old
in st.probed_points):
            continue
        if agent._clear_decision(st) is not None:
            continue # 已可清除的源不必再补测
        samples = list(agent._snapshot(st).samples)
        if not samples:
            continue
        receive = [target for target in samples if math.dist(point, target) <=
MIN_RECEIVE_R]
        p_rec = len(receive) / len(samples)
        if p_rec < MIN_P_REC:
            continue

```

```

        quality = sum(max((math.sin(math.radians(
            wrap(bearing(station, target) - bearing(point, target)))) ** 2
            for station, _ in st.observations), default=0.0)
            for target in receive) / len(receive)
        value_per_second = p_rec * quality / MEASURE_SECONDS
        if value_per_second >= VALUE_PER_SECOND_MIN:
            choices.append((value_per_second, channel))
    for _, channel in sorted(choices, reverse=True)[:max_coobservations]:
        agent.observe(point, channel)
        agent.state[channel].attempts += 1
        agent.co_measurements += 1

```

q3_coverage_ring.py

以最坏覆盖距离闭式解联合设计覆盖环点数 k 与半径 R ，解析保证任务圆域内每个源都被某个停靠点覆盖。

```

from __future__ import annotations

import math

AREA_R = 1800.0          # 任务圆域半径 (m)
MIN_RECEIVE_R = 1000.0  # 接收半径下界；检测必须  $d \leq 1000$  m 才能保证有信号
RING_COUNT = 7          # J+ 采用的环点数  $k$ 
SEARCH_RING_R = 1000.0  # J+ 采用的环半径  $R$  (m)

def worst_coverage_distance(k: int, ring_r: float) -> float:
    """ $k$  个等角环点、半径  $R$  时的最坏覆盖距离。

    最坏点落在任务圆域边界上相邻环点的角平分线处：
     $d_k(R) = \sqrt{1800^2 + R^2 - 2 \cdot 1800 \cdot R \cdot \cos(\pi/k)}$ 。
    """
    return math.sqrt(AREA_R ** 2 + ring_r ** 2 - 2.0 * AREA_R * ring_r *
        math.cos(math.pi / k))

def min_safe_radius(k: int) -> float:
    """由  $d_k(R) \leq 1000$  解出的最小安全半径  $R_{\min}(k)$ 。

     $R^2 - 3600 \cdot \cos(\pi/k) \cdot R + 2240000 \leq 0$ ，取较小根：
     $R_{\min}(k) = 1800 \cdot \cos(\pi/k) - \sqrt{3240000 \cdot \cos^2(\pi/k) - 2240000}$ 。
    """
    cosine = math.cos(math.pi / k)
    return AREA_R * cosine - math.sqrt(3240000.0 * cosine * cosine - 2240000.0)

def open_sweep_length(k: int, ring_r: float) -> float:
    """原点 +  $k$  个环点的开环巡回长度： $R \cdot (1 + 2(k-1) \cdot \sin(\pi/k))$ 。"""
    return ring_r * (1.0 + 2.0 * (k - 1) * math.sin(math.pi / k))

def search_points(k: int = RING_COUNT, ring_r: float = SEARCH_RING_R):
    """覆盖发现层的停靠点集合：原点（覆盖圆心附近）+  $k$  个等角外环点。

    每多一个环点要多扫一轮未发现频道（约 60 s）： $k=6 \rightarrow 7$  省 106 s 移动只付约 61 s
    扫描； $k=7 \rightarrow 8$  只省 49 s 移动却仍付约 60 s，故联合最优取  $k=7$ 、 $R \approx 1000$ 。
    """
    step = 360.0 / k
    return [(0.0, 0.0)] + [
        (ring_r * math.cos(math.radians(step * i)), ring_r *
        math.sin(math.radians(step * i)))
        for i in range(k)
    ]

```

```

]

if __name__ == "__main__":
    for k in (6, 7, 8):
        print(f"k={k}: R_min={min_safe_radius(k):8.1f} m      "
              f"d_k(1000)={worst_coverage_distance(k, 1000.0):8.2f} m      "
              f"巡回(R=1000)={open_sweep_length(k, 1000.0):7.0f} m")
    print(f"J+ 采用 k={RING_COUNT}, R={SEARCH_RING_R:.0f} m: "
          f"最坏覆盖 {worst_coverage_distance(RING_COUNT, SEARCH_RING_R):.2f} m"
          f" ( 余量 {MIN_RECEIVE_R - worst_coverage_distance(RING_COUNT,
SEARCH_RING_R):.2f} m) , "
          f"巡回 {open_sweep_length(RING_COUNT, SEARCH_RING_R):.0f} m")

```

q3_feasible_region.py

用 $\pm 1^\circ$ 方位带与距离上界裁剪出保守可行域，并以最小包围圆判定能否前往圆心清除。

```

from __future__ import annotations

import math

AREA_R = 1800.0          # 任务圆域半径
MAX_DIRECTION_R = 1500.0 # 接收半径上界: 超过必然无信号, 可作裁剪约束
GUARANTEE_R = 1000.0     # 接收半径下界: 环内必然有信号
ERROR_DEG = 1.0          # 示向度误差半角
CLEAR_RADIUS_M = 19.75   # 20 m 光学/清除半径留 0.25 m 数值余量

def clip(poly, a, b, c):
    """Sutherland-Hodgman 裁剪一条半平面  $a*x + b*y \leq c$ , 返回交点多边形。"""
    if not poly:
        return []
    out, previous = [], poly[-1]
    previous_value = a * previous[0] + b * previous[1] - c
    for current in poly:
        current_value = a * current[0] + b * current[1] - c
        previous_inside = previous_value <= 1e-8
        current_inside = current_value <= 1e-8
        if previous_inside != current_inside:
            t = previous_value / (previous_value - current_value)
            out.append((previous[0] + t * (current[0] - previous[0]),
                        previous[1] + t * (current[1] - previous[1])))
        if current_inside:
            out.append(current)
        previous, previous_value = current, current_value
    return out

def clip_bearing(poly, station, angle_deg):
    """加入一条  $\pm 1^\circ$  示向度带 (张角  $2^\circ$  的闭楔形), 即两个半平面约束。"""
    lower = math.radians(angle_deg - ERROR_DEG)
    upper = math.radians(angle_deg + ERROR_DEG)
    for a, b in ((math.sin(lower), -math.cos(lower)),
                 (-math.sin(upper), math.cos(upper))):
        poly = clip(poly, a, b, a * station[0] + b * station[1])
    return poly

def task_area_polygon(sides=72):

```

```

        """任务圆域的保守表示：外接正 sides 边形（默认 72 边）。"""
        vertex_radius = AREA_R / math.cos(math.pi / sides)
        return [(vertex_radius * math.cos(2 * math.pi * (k + .5) / sides),
                  vertex_radius * math.sin(2 * math.pi * (k + .5) / sides)) for k in
range(sides)]

def clip_outer_circle(poly, center, radius, sides=72):
    """用外接正 sides 边形保守表示距离上界 d <= radius (72 面切向在实现中预缓存)。"""
    if not poly:
        return []
    cx, cy = center
    limit = radius * radius + 1e-9
    if all((x - cx) ** 2 + (y - cy) ** 2 <= limit for x, y in poly):
        return poly # 全部顶点已在半径内，整段裁剪可跳过
    for k in range(sides):
        a, b = math.cos(2 * math.pi * k / sides), math.sin(2 * math.pi * k /
sides)
        poly = clip(poly, a, b, radius + a * cx + b * cy)
        if not poly:
            break
    return poly

def _circle_two(a, b):
    center = ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)
    return center, math.dist(center, a)

def _circle_three(a, b, c):
    d = 2 * (a[0] * (b[1] - c[1]) + b[0] * (c[1] - a[1]) + c[0] * (a[1] - b[1]))
    if abs(d) < 1e-10:
        return None
    aa, bb, cc = a[0] ** 2 + a[1] ** 2, b[0] ** 2 + b[1] ** 2, c[0] ** 2 + c[1] **
2
    center = ((aa * (b[1] - c[1]) + bb * (c[1] - a[1]) + cc * (a[1] - b[1])) / d,
              (aa * (c[0] - b[0]) + bb * (a[0] - c[0]) + cc * (b[0] - a[0])) / d)
    return center, math.dist(center, a)

def minimum_enclosing_circle(points):
    """随机增量最小包围圆：圆心是 0/1/2/3 个边界点的确定结果，半径即包围圆半径。"""
    if not points:
        return (0.0, 0.0), math.inf
    circle = (points[0], 0.0)
    for i, p in enumerate(points):
        if math.dist(p, circle[0]) <= circle[1] + 1e-7:
            continue
        circle = (p, 0.0)
        for j, q in enumerate(points[:i]):
            if math.dist(q, circle[0]) <= circle[1] + 1e-7:
                continue
            circle = _circle_two(p, q)
            for r in points[:j]:
                if math.dist(r, circle[0]) <= circle[1] + 1e-7:
                    continue
                candidate = _circle_three(p, q, r)
                if candidate is not None:
                    circle = candidate
    return circle

def _inside_convex(poly, point):
    return all((poly[(i + 1) % len(poly)][0] - poly[i][0]) * (point[1] -
poly[i][1])
               - (poly[(i + 1) % len(poly)][1] - poly[i][1]) * (point[0] -
poly[i][0]) >= -1e-7
               for i in range(len(poly)))

```

```

def belief_samples(poly, limit=64):
    """在可行域上取 11×11 网格内点并补边界顶点，薄区域也不会被网格漏掉。"""
    if not poly:
        return []
    xs, ys = [p[0] for p in poly], [p[1] for p in poly]
    points = [(min(xs) + (max(xs) - min(xs)) * (i + .5) / 11,
                min(ys) + (max(ys) - min(ys)) * (j + .5) / 11)
               for i in range(11) for j in range(11)]
    points = [p for p in points if _inside_convex(poly, p)] + list(poly)
    if len(points) <= limit:
        return points
    step = len(points) / limit
    return [points[min(int(k * step), len(points) - 1)] for k in range(limit)]

def build_region(observations, no_signal_points):
    """信念层主流程：任务圆域外接 72 边形起，逐条观测裁剪方位带与 1500 m 上界。

    返回（保守多边形，样本，接收半径区间）；no_signal 排除圆是非凸孔洞，
    只从样本里删除，保守多边形仍用于清除判定以免误清除。
    """
    poly = task_area_polygon()
    for station, angle in observations:
        poly = clip_bearing(poly, station, angle)
        poly = clip_outer_circle(poly, station, MAX_DIRECTION_R)
        if not poly:
            break
    samples, bounds = [], []
    for point in belief_samples(poly):
        lower = max([GUARANTEE_R] + [math.dist(point, station) for station, _ in
observations])
        upper = min([MAX_DIRECTION_R] + [math.dist(point, station) for station in
no_signal_points])
        if upper - lower > 1e-9:
            samples.append(point)
            bounds.append((lower, upper))
    return poly, samples, bounds

def minimum_enclosing_circle_of_region(poly):
    return minimum_enclosing_circle(list(poly))

def clear_decision(poly):
    """清除判定：保守外包络的最小包围圆半径不超过 19.75 m 时才前往圆心清除。"""
    center, radius = minimum_enclosing_circle_of_region(poly)
    return center if radius <= CLEAR_RADIUS_M else None

def reception_probabilities(samples, bounds, station):
    """固定未知接收半径下的条件接收概率 p_rec，供四指标 p_rec 项使用。"""
    probabilities = []
    for target, (lower, upper) in zip(samples, bounds):
        distance = math.dist(target, station)
        if distance <= lower:
            probabilities.append(1.0)
        elif distance >= upper:
            probabilities.append(0.0)
        else:
            probabilities.append((upper - distance) / (upper - lower))
    return probabilities

def predicted_circle(poly, station, predicted_bearing):
    """预加入一条假设方位后的包围圆，用于四指标中的 R_pred 项。"""
    updated = clip_bearing(list(poly), station, predicted_bearing)
    updated = clip_outer_circle(updated, station, MAX_DIRECTION_R)
    return minimum_enclosing_circle(updated)

```

q3_scheduler.py

逐动作滚动清除调度，配合覆盖点顺序枚举与最近邻+2-opt/Or-opt 开放路径重排实现全清除。

```
from __future__ import annotations

import math
from itertools import permutations

def two_opt_route(start, anchors):
    """从当前位置 start 出发的开放路径：最近邻初解，再用 2-opt 与 Or-opt 交替局部优化。

    2-opt 只反转连续段，表达不了"把被甩到路线末尾的点搬回其所在簇"的纯插入移动；
    Or-opt 补足该邻域：把长度 1-3 的段正向或反向插入其它位置。
    """
    if len(anchors) < 2:
        return list(anchors)
    remaining, route, current = set(anchors), [], start
    while remaining:
        nxt = min(remaining, key=lambda c: math.dist(current, anchors[c]))
        route.append(nxt)
        current = anchors[nxt]
        remaining.remove(nxt)

    def route_cost(order):
        points = [start] + [anchors[c] for c in order]
        return sum(math.dist(a, b) for a, b in zip(points, points[1:]))

    improved = True
    while improved:
        improved = False
        old_cost = route_cost(route)
        for i in range(len(route) - 1):
            for j in range(i + 2, len(route) + 1):
                candidate = route[:i] + list(reversed(route[i:j])) + route[j:]
                if route_cost(candidate) + 1e-6 < old_cost:
                    route, improved = candidate, True
                    break
            if improved:
                break
        if improved:
            continue
        old_cost, n = route_cost(route), len(route)
        for seg_len in (1, 2, 3):
            for i in range(n - seg_len + 1):
                segment = route[i:i + seg_len]
                rest = route[:i] + route[i + seg_len:]
                for j in range(len(rest) + 1):
                    for seg in (segment, segment[::-1]):
                        candidate = rest[:j] + seg + rest[j:]
                        if route_cost(candidate) + 1e-6 < old_cost:
                            route, improved = candidate, True
                            break
                    if improved:
                        break
                if improved:
                    break
            if improved:
                break
        if improved:
            break
    return route

def coverage_order(position, unvisited, anchors):
```

```

"""覆盖点访问顺序枚举（点数很少）：路程 + 终点到最近已发现目标锚点的衔接距离。"""
if len(unvisited) <= 1:
    return list(unvisited)
best_order, best_cost = None, math.inf
for order in permutations(unvisited):
    path = [position, *order]
    cost = sum(math.dist(a, b) for a, b in zip(path, path[1:]))
    # 在保证覆盖路线短的同时，选一个对已发现目标更有用的收尾侧。
    if anchors:
        cost += min(math.dist(order[-1], anchor) for anchor in anchors)
    if cost < best_cost - 1e-6:
        best_order, best_cost = order, cost
return list(best_order) if best_order is not None else list(unvisited)

def rolling_clear(agent, max_steps: int = 240):
    """模型 G/J+ 的清除阶段主循环：每一步只执行路线上首个目标的一次动作，随后重规划。

    每步用最近邻 + 2-opt/0r-opt 重排全部未清除源，只执行首个目标的一次清除或一次测量，
    更新信念后立即重规划；词序目标先保证全清除，再压缩平均与 P95 每源耗时。
    """
    for _ in range(max_steps):
        unresolved = [channel for channel, st in agent.state.items()
                       if st.discovered and not st.cleared and not st.exhausted]
        if not unresolved:
            break
        channel = agent._two_opt_route(unresolved)[0]
        st = agent.state[channel]
        center = agent._clear_decision(st)
        if center is not None:
            if agent._try_clear(center, channel):
                st.cleared = True
                agent._coobserve(center, channel)
                continue
        if st.attempts >= 10:
            st.exhausted = True
            continue
        point = agent._next_measurement_point(st)
        if point is None:
            st.exhausted = True
            continue
        agent.observe(point, channel)
        st.attempts += 1
        agent._coobserve(point, channel)
    return agent.summary()

```

q3_selector.py

第二问四指标选点内核——粗网格到加密两阶段打分，并在高分区内选出下一测量点。

```
from __future__ import annotations

import math
from dataclasses import dataclass

COARSE_DISTANCES = (200.0, 600.0, 1000.0, 1400.0, 1800.0) # 以首次检测站为原点的粗
网格距离
COARSE_BEARING_STEP = 60.0
GOOD_ETA = 0.10 # 高分区带宽：峰值的 10% (range-based, 在线负分时仍有效)
SPEED = 5.0 # 移动速度 5 m/s
MEASURE_TIME_S = 6.0 # 一次测量动作 (切换 1 s + 检测 5 s)
MAX_EXTRA_DISTANCE_M = 250.0 # C+ 限制：绕行测量相对直奔圆心的额外路程上限

WEIGHTS_C = (0.35, 0.30, 0.15, 0.20) # C: 第二问示例权重
WEIGHTS_CPLUS = (0.25, 0.25, 0.35, 0.15) # C+: 第三问在线权重
WEIGHTS_JPLUS = (0.25, 0.25, 0.25, 0.25) # J+: 等权标定 (最终模型)

@dataclass(frozen=True)
class SelectionDecision:
    position: tuple[float, float]
    score: float
    q_geo: float
    p_rec: float
    travel_s: float
    r_pred_m: float

def add(point, length, angle_deg):
    angle = math.radians(angle_deg)
    return (point[0] + length * math.cos(angle), point[1] + length *
math.sin(angle))

def bearing(a, b):
    return math.degrees(math.atan2(b[1] - a[1], b[0] - a[0]))

def wrap(angle_deg):
    return (angle_deg + 180.0) % 360.0 - 180.0

def normal(values, value):
    """min-max 归一化：上界等于下界时取 1.0。"""
    lo, hi = min(values), max(values)
    return 1.0 if hi <= lo else (value - lo) / (hi - lo)

def raw_metrics(samples, observations, current_position, candidate, min_receive_r,
predicted_radius, third="completion"):
    """四项原始指标 q_geo、p_rec、travel_s、r_pred：候选点看不到任何样本时返回 None。

    q_geo：对全部信念样本取「与历史各站交角  $\sin^2$ 」的最大值再平均，衡量交会几何质量；
    p_rec：候选点在保证接收半径内的样本占比；
    third="move" 第三指标为到候选点的移动时间 (C 口径)；
    third="completion" 第三指标为完成时间 = (到候选点 + 候选点到预测圆心)/v + 6 s (C+
起口径)。
    """
    if not samples:
        return None
    receive = [target for target in samples if math.dist(candidate, target) <=
min_receive_r]
    if not receive:
```

```

        return None
    qualities = []
    for target in samples:
        candidate_angle = bearing(candidate, target)
        qualities.append(max((math.sin(math.radians(wrap(bearing(station, target)
- candidate_angle))) ** 2
                                for station, _ in observations), default=0.0))
    radii, centers = [], []
    for target in receive:
        center, radius = predicted_radius(candidate, bearing(candidate, target))
        if math.isfinite(radius):
            radii.append(radius)
            centers.append(center)
    if not radii:
        return None
    q_geo = sum(qualities) / len(qualities)
    p_rec = len(receive) / len(samples)
    r_pred = sum(radii) / len(radii)
    if third == "move":
        return q_geo, p_rec, math.dist(current_position, candidate) / SPEED,
r_pred
    post_distance = sum(math.dist(candidate, center) for center in centers) /
len(centers)
    path_distance = math.dist(current_position, candidate) + post_distance
    return q_geo, p_rec, path_distance / SPEED + MEASURE_TIME_S, r_pred

def score_pool(pool, weights, metrics_fn):
    """池内四项指标各自 min-max 归一化后加权: J = w1·q_geo + w2·p_rec - w3·travel_s -
w4·r_pred. """
    rows = [(q, *metrics_fn(q)) for q in pool]
    rows = [(q, *m) for q, *m in rows if m[0] is not None]
    if not rows:
        return []
    columns = [[row[i] for row in rows] for i in range(1, 5)]
    scored = []
    for q, q_geo, p_rec, travel_s, r_pred in rows:
        normalized = [normal(columns[i], value)
                        for i, value in enumerate((q_geo, p_rec, travel_s, r_pred))]
        score = (weights[0] * normalized[0] + weights[1] * normalized[1]
                 - weights[2] * normalized[2] - weights[3] * normalized[3])
        scored.append(SelectionDecision(q, score, q_geo, p_rec, travel_s, r_pred))
    return scored

def select_from_scored(scored):
    """高分区 (range-based R_good 带) 内优先短移动, 其次取预测包围圆更小的点. """
    peak = max(item.score for item in scored)
    low = min(item.score for item in scored)
    threshold = peak - GOOD_ETA * (peak - low)
    return min((item for item in scored if item.score >= threshold),
                key=lambda item: (item.travel_s, item.r_pred_m, -item.score))

def next_measurement_point(station, bearing_deg, weights, metrics_fn):
    """选点主流程: 粗网格 (5 距离 × 6 方位) 取前二 → ±100 m、±10° 邻域加密 → 重算分数。

    加密集上重新归一化与评分 (与第二问求解器完全一致), 最后选执行点。
    """
    coarse_points = [add(station, d, bearing_deg + offset)
                      for d in COARSE_DISTANCES for offset in range(0, 360,
int(COARSE_BEARING_STEP))]
    coarse = score_pool(coarse_points, weights, metrics_fn)
    if not coarse:
        return None
    seeds = sorted(coarse, key=lambda item: item.score, reverse=True)[:2]
    refined_points, seen = [], set()

```

```

    for item in seeds:
        distance = math.dist(station, item.position)
        base = bearing(station, item.position)
        for dd in (max(50.0, distance - 100.0), distance, min(1800.0, distance +
100.0)):
            for da in (-10.0, 0.0, 10.0):
                point = add(station, dd, base + da)
                key = (round(point[0], 9), round(point[1], 9))
                if key not in seen:
                    seen.add(key)
                    refined_points.append(point)
            refined = score_pool(refined_points, weights, metrics_fn) or coarse
    return select_from_scored(refined)

```

q4_exact_verify.py

核对 Q4 最终 D 模型的 B/C 路线资产与距离门边界不变量

```

from __future__ import annotations

import json
import math
from pathlib import Path

from q4_solver import select_route, should_coobserve

HERE = Path(__file__).resolve().parent
ASSETS = HERE.parent / "assets" / "d_model"

def load_route(filename: str) -> list[tuple[float, float]]:
    payload = json.loads((ASSETS / filename).read_text(encoding="utf-8"))
    sites = [tuple(map(float, point)) for point in payload["sites"]]
    return [sites[index] for index in payload["order"]]

def verify() -> dict[str, object]:
    routes = {
        "B": load_route("b_winner_design.json"),
        "C": load_route("c_winner_design.json"),
    }
    assert len(routes["B"]) == 27 and len(routes["C"]) == 29
    assert all(route[0] == (0.0, 0.0) for route in routes.values())
    assert all(len(set(route)) == len(route) for route in routes.values())
    assert select_route(0) == "B" and all(select_route(n) == "C" for n in (1, 2,
3))

    region = ((-1.0, -1.0), (1.0, -1.0), (1.0, 1.0), (-1.0, 1.0))
    assert should_coobserve((1501.0, 0.0), region)
    assert not should_coobserve((1501.1, 0.0), region)

    return {
        name: {
            "stations": len(route),
            "max_radius_m": round(max(math.hypot(*point) for point in route), 3),
        }
        for name, route in routes.items()
    }

```

```
if __name__ == "__main__":
    print(verify())
```

q4_solver.py

实现 Q4 最终 D 模型的原点分支门控与 1500 米保守共观测距离门。

```
from __future__ import annotations

import math
from collections.abc import Sequence

Point = tuple[float, float]
MAX_RECEPTION_RADIUS_M = 1500.0

def select_route(origin_discoveries: int) -> str:
    """原点零发现走 B 路线，否则走 C 路线。"""
    if origin_discoveries < 0:
        raise ValueError("origin_discoveries must be nonnegative")
    return "B" if origin_discoveries == 0 else "C"

def point_segment_distance(point: Point, a: Point, b: Point) -> float:
    dx, dy = b[0] - a[0], b[1] - a[1]
    denominator = dx * dx + dy * dy
    if denominator <= 1e-12:
        return math.dist(point, a)
    t = ((point[0] - a[0]) * dx + (point[1] - a[1]) * dy) / denominator
    t = max(0.0, min(1.0, t))
    projection = (a[0] + t * dx, a[1] + t * dy)
    return math.dist(point, projection)

def point_in_convex_polygon(point: Point, polygon: Sequence[Point]) -> bool:
    signs: list[bool] = []
    for index, a in enumerate(polygon):
        b = polygon[(index + 1) % len(polygon)]
        cross = ((b[0] - a[0]) * (point[1] - a[1])
                 - (b[1] - a[1]) * (point[0] - a[0]))
        if abs(cross) > 1e-8:
            signs.append(cross > 0.0)
    return not signs or all(sign == signs[0] for sign in signs)

def distance_to_feasible_region(station: Point, polygon: Sequence[Point]) -> float:
    """计算测站到保守凸可行域的欧氏最小距离。"""
    if not polygon or point_in_convex_polygon(station, polygon):
        return 0.0
    return min(
        point_segment_distance(station, a, polygon[(i + 1) % len(polygon)])
        for i, a in enumerate(polygon)
    )

def should_coobserve(station: Point, feasible_polygon: Sequence[Point]) -> bool:
    """等号保留；仅剪去距离严格大于 1500 米的不可能成功测量。"""
    return distance_to_feasible_region(station, feasible_polygon) <= MAX_RECEPTION_RADIUS_M

if __name__ == "__main__":
    square = ((-1.0, -1.0), (1.0, -1.0), (1.0, 1.0), (-1.0, 1.0))
    print({
        "gate": {n: select_route(n) for n in range(4)},
```

```
    "1499m": should_coobserve((1500.0, 0.0), square),  
    "1500m": should_coobserve((1501.0, 0.0), square),  
    "1500.1m": should_coobserve((1501.1, 0.0), square),  
  })
```